UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA UPCA

Sistema Inteligente de Comparação de Preços e Alertas de E-Commerce

Pipeline ETL com KNIME, Node-RED, PostgreSQL e Grafana

Trabalho Prático I – Integração de Sistemas de Informação

Hugo Manuel Alves Gomes - 31545 - Regime Pós-laboral

Orientação

Luís Ferreira

Ano letivo 2026/2027

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

Índice

Resumo 1
Abstract 2
Siglas e acrónimos 3
Glossário 4
1. Introdução 5
1.1 Contexto 5
1.2 Objetivos 6
1.3 Estrutura do documento 6
2. Problema 7
3. Estratégia utilizada 8
3.1 Requisitos 8
3.2 Arquitetura 9
3.3 Fontes de dados 10
3.4 Tecnologias e justificação 12
3.5 Princípios de desenho 14
4. Transformações 15
4.1 Uniformização das quatro lojas 17
4.2 Limpeza com expressões regulares 19
4.3 Validação 21
4.4 Enriquecimento com joins 24
4.5 Cálculo do custo total 26
4.6 Melhor loja, poupança e alertas 28
5. Jobs 30
5.1 Carregamento na base de dados 32
5.2 Alertas por e-mail, sem repetições 36
5.3 Exportação e transferência por SFTP 38
5.4 Tratamento de erros e logs 40
5.5 Jobs no Node-RED 42
5.6 Execução em batch e agendamento 45
6. Vídeo de demonstração 47
7. Conclusão e trabalhos futuros 48
7.1 Conclusões 48
7.2 Limitações e trabalhos futuros 49
8. Referências 50

Índice de figuras

Figura 1 Arquitetura do sistema. Fonte: elaboração própria. 9
Figura 2 Visão geral do workflow 01_extracao, organizado por zonas. 16
Figura 3 Extração e uniformização das quatro lojas. 18
Figura 4 Preços e portes da loja UK antes e depois da limpeza. 20
Figura 5 Cálculo do EAN 21
Figura 6 Linhas rejeitadas pela validação, com o respetivo motivo. 22
Figura 7 Enriquecimento: obtenção das tabelas de referência (configuração, VATComply, Frankfurter e Fake Store API) e junções com os preços. 24
Figura 8 Resultado do left outer join entre as lojas e os países da VATComply. 25
Figura 9 Cálculo do custo total com sete nós Math Formula e arredondamento final. 26
Figura 10 Cálculo do custo 26
Figura 11 Cálculo dos direitos 26
Figura 12 Custo total da Mochila Fjallraven nas quatro lojas. 27
Figura 13 Determinação da melhor loja, cálculo da poupança, alertas e exportação dos resultados. 28
Figura 14 Alertas gerados: clientes cujo preço-alvo foi atingido. 29
Figura 15 Sequência dos jobs em cada execução; a destaque, a regra das 24 horas. 31
Figura 16 Modelo de dados da base de dados PostgreSQL. 32
Figura 17 Carregamento na base de dados: DB Merger, DB Row Inserter e cadeia de variáveis de fluxo. 33
Figura 18 Tentativas de DELETE e DROP TABLE recusadas pela base de dados. 35
Figura 19 Job de alertas por e-mail, com o ramo de envio inativo por não existirem alertas novos. 36
Figura 20 E-mails de alerta recebidos no Mailpit. 37
Figura 21 Job de transferência por SFTP, protegido por um bloco Try/Catch. 38
Figura 22 Ficheiros recebidos na pasta /upload do servidor SFTP. 39
Figura 23 Registo da execução na base de dados e no ficheiro de log. 40
Figura 24 Painel de logs no Grafana, com execuções OK e ERRO. 41
Figura 25 Fluxos do Node-RED: (1) conversão da configuração, (2) simulação da loja US e (3) endpoint para o KNIME. 42
Figura 26 Excerto da resposta do endpoint /loja-us/precos, com os preços no formato americano. 44
Figura 27 Agendamento do workflow no Node-RED, com o resultado de uma execução. 46
Figura 28 QR Code de acesso ao vídeo de demonstração. 47

Índice de tabelas

Tabela 1Requisitos funcionais. 8
Tabela 2 Requisitos não funcionais 9
Tabela 3 Fontes de dados, formatos e origem. 10
Tabela 4 Tecnologias escolhidas e justificação. 12
Tabela 5 Evolução do número de registos de preços numa execução. 17
Tabela 6 Mapeamento dos campos de cada loja para o esquema comum. 17
Tabela 7 Funções e expressões utilizadas na limpeza dos preços, portes e endereços de e-mail. 19
Tabela 8 Registos rejeitados numa execução e respetivo motivo. 23
Tabela 9 Junções realizadas no enriquecimento dos preços. 24
Tabela 10 Variáveis de fluxo usadas no cálculo do custo total. 25
Tabela 11 Expressões dos nós Math Formula do cálculo do custo total. 26
Tabela 12 Cálculo do custo total da Mochila Fjallraven (execução de 22 de outubro de 2026). 27
Tabela 13 Melhor loja e poupança por produto (execução de 22 de outubro de 2026). 28
Tabela 14 Comparação entre o melhor custo total e o preço-alvo de três clientes. 29
Tabela 15 Jobs implementados, ferramenta e secção correspondente. 30
Tabela 16 Nós de carregamento e comportamento de cada tabela. 33
Tabela 17 Configuração do nó Transfer Files. 38
Tabela 18 Testes realizados e resultados obtidos. 41
Tabela 19 Códigos de saída do NodePit Batch. 45
Tabela 20 Testes da execução em batch e do agendamento. 46

Resumo

Com a multiplicação das lojas online, o preço de etiqueta deixou de indicar, por si só, qual é a loja mais barata. Numa compra internacional somam-se portes, conversões de moeda, direitos aduaneiros, IVA de importação e taxas de desalfandegamento. Este trabalho apresenta um sistema de integração de dados que calcula o custo total real de cada produto em lojas de quatro países (Portugal, Espanha, Reino Unido e Estados Unidos) e avisa os utilizadores quando esse custo atinge ou desce abaixo de um preço-alvo definido.

A solução segue um processo ETL desenvolvido no KNIME. Os dados são extraídos de ficheiros CSV, JSON, XML e YAML, de um stream simulado no Node-RED e de três APIs REST públicas (catálogo, câmbios e taxas de IVA). Na transformação, os preços são normalizados com expressões regulares, os códigos EAN-13 e os e-mails são validados, as fontes são cruzadas com joins e as regras aduaneiras são aplicadas. No carregamento, os resultados são gravados em PostgreSQL, enviados por e-mail e por SFTP, registados em logs e apresentados num dashboard Grafana. Um job orquestra estas tarefas com tratamento de erros e corre de hora a hora em modo batch.

Os resultados mostram que a loja com o preço de etiqueta mais baixo nem sempre é a mais barata. Um produto com preço de £94,56 no Reino Unido custa 161,01 € depois de importado, contra 106,65 € em Espanha. Em cada execução, dos 35 registos recebidos, 1 duplicado é removido e 4 são rejeitados com o motivo registado. O sistema garante ainda que os dados nunca são eliminados e que os alertas não se repetem dentro de 24 horas.

Palavras-chave: ETL, integração de dados, KNIME, Node-RED, comparação de preços, expressões regulares.

Abstract

With the growth of online shops, the price tag alone no longer shows which shop is the cheapest. An international purchase adds shipping costs, currency conversion, customs duties, import VAT and customs clearance fees. This work presents a data integration system that calculates the real total cost of each product in shops from four countries (Portugal, Spain, the United Kingdom and the United States) and alerts users when that cost reaches or falls below a target price they have set.

The solution follows an Extract, Transform, Load (ETL) process developed in KNIME. Data is extracted from CSV, JSON, XML and YAML files, from a stream simulated in Node-RED and from three public REST APIs (product catalogue, exchange rates and VAT rates). In the transformation stage, prices are normalised with regular expressions, EAN-13 codes and e-mail addresses are validated, sources are combined with joins and customs rules are applied. In the load stage, the results are stored in PostgreSQL, sent by e-mail and SFTP, recorded in logs and displayed on a Grafana dashboard. A job orchestrates these tasks with error handling and runs every hour in batch mode.

The results show that the shop with the lowest tag price is not always the cheapest. A product priced at £94.56 in the United Kingdom costs €161.01 once imported, compared with €106.65 in Spain. In each run, out of 35 records received, 1 duplicate is removed and 4 are rejected with the reason recorded. The system also ensures that data is never deleted and that alerts are not repeated within 24 hours.

Keywords: ETL, data integration, KNIME, Node-RED, price comparison, regular expressions.

Siglas e acrónimos

API - Application Programming Interface (Interface de Programação de Aplicações)

BCE - Banco Central Europeu

BD - Base de Dados

CSV - Comma-Separated Values (Valores Separados por Vírgulas)

EAN - European Article Number (Número Europeu de Artigo)

ETL - Extract, Transform, Load (Extração, Transformação e Carregamento)

HTTP - HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)

IA - Inteligência Artificial

IVA - Imposto sobre o Valor Acrescentado

JDBC - Java Database Connectivity (Conectividade Java a Bases de Dados)

JSON - JavaScript Object Notation (Notação de Objetos JavaScript)

KNIME - Konstanz Information Miner

REST - Representational State Transfer (Transferência de Estado Representacional)

SFTP - SSH File Transfer Protocol (Protocolo de Transferência de Ficheiros sobre SSH)

SMTP - Simple Mail Transfer Protocol (Protocolo Simples de Transferência de Correio)

SQL - Structured Query Language (Linguagem de Consulta Estruturada)

SSH - Secure Shell

UE - União Europeia

UTC - Coordinated Universal Time (Tempo Universal Coordenado)

XML - eXtensible Markup Language (Linguagem de Marcação Extensível)

YAML - YAML Ain't Markup Language

Glossário

Agendamento – Execução automática sem a intervenção de humanos de um processo em intervalos fixos.

Batch (execução em) – Execução de um workflow a partir da linha de comandos, sem abrir a interface gráfica da ferramenta.

Contentor – Ambiente isolado e reproduzível, criado pelo Docker, onde corre um serviço com as suas dependências.

Custo total importado – Preço de um produto somado aos portes, convertido para euros e acrescido dos direitos aduaneiros, do IVA de importação e da taxa de desalfandegamento.

Dígito de controlo – Último dígito de um código EAN-13, calculado a partir dos 12 anteriores, que permite detetar erros de leitura ou de escrita.

Dump – Ficheiro SQL com a estrutura e os dados completos de uma base de dados, que permite recriá-la noutro sistema.

Expressão regular – Padrão de texto usado para procurar, validar ou substituir sequências de caracteres.

Junction – Ligação de pasta do sistema de ficheiros do Windows que faz uma pasta aparecer noutro local sem copiar o conteúdo.

Stream – Fluxo contínuo de dados produzidos ao longo do tempo, em vez de um ficheiro fechado.

Trigger – Função da base de dados executada automaticamente antes ou depois de uma operação sobre uma tabela.

Upsert – Operação que atualiza um registo se ele já existir e o insere caso contrário (nó DB Merger, no KNIME).

Workflow Data Area – Pasta data de um workflow KNIME, a única pasta interna do workflow a que os nós podem aceder.

1. Introdução

As organizações raramente têm os seus dados num único sistema. Fornecedores, parceiros e serviços externos disponibilizam informação em formatos, estruturas e convenções diferentes, e muitas vezes com erros. A integração de sistemas de informação ao nível dos dados resolve este problema, reúne dados de fontes heterogéneas, uniformiza-os e disponibiliza-os de forma fiável a quem deles precisa.

Um dos padrões mais usado para este fim é o processo de Extração, Transformação e Carregamento (Extract, Transform, Load – ETL). Na extração, os dados são recolhidos das fontes de origem, sejam ficheiros, bases de dados, serviços web ou fluxos contínuos. Na transformação, são limpos, validados, normalizados, cruzados e enriquecidos segundo as regras de negócio. No carregamento, os resultados são entregues aos sistemas de destino como bases de dados, ficheiros de exportação, notificações e dashboards. Um processo ETL completo inclui ainda jobs que orquestram estas tarefas, controlam a ordem de execução, tratam de falhas e registam o que aconteceu.

O comércio eletrónico é um ótimo exemplo da necessidade de integração, cada loja publica preços no seu formato, na sua moeda e com as suas condições de envio, e comparar lojas de países diferentes exige juntar e transformar todos esses dados.

1.1 Contexto

Este relatório documenta o Trabalho Prático I da Unidade Curricular de Integração de Sistemas de Informação (ISI), da Licenciatura em Engenharia de Sistemas Informáticos, no ano letivo 2026/2027. O enunciado pede o desenvolvimento de processos ETL estruturados em três fases obrigatórias que utilizem fontes e formatos diversos, interliguem várias tecnologias e incluam *jobs* com acesso a serviços remotos.

O trabalho foi realizado individualmente. O cenário escolhido é simulado, mas usa interfaces de programação de aplicações (APIs) reais, um catálogo de produtos, taxas de câmbio do Banco Central Europeu, taxas de IVA dos países da União Europeia e regras aduaneiras inspiradas nas aplicáveis às compras feitas fora da UE. A ferramenta principal é o KNIME Analytics Platform, complementado pelo Node-RED, pelo PostgreSQL, pelo Grafana e por serviços de correio eletrónico e de transferência de ficheiros executados em contentores Docker.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver um processo ETL completo que, a partir de dados de lojas de quatro países, determine o custo total real de cada produto e informe os utilizadores das melhores oportunidades de compra. Pretende-se, ao mesmo tempo, demonstrar a capacidade de interligar diferentes tecnologias ao longo de todo o ciclo de vida dos dados, desde a recolha até à visualização. Este objetivo geral divide-se nos seguintes objetivos específicos:

1. Extrair dados de fontes heterogéneas: ficheiros CSV (Comma-Separated Values), JSON (JavaScript Object Notation), XML (eXtensible Markup Language) e YAML (YAML Ain't Markup Language), um *stream* de dados e APIs REST.
2. Limpar, validar e normalizar os dados, recorrendo a expressões regulares.
3. Calcular o custo total real de cada produto em cada loja, incluindo portes, câmbio e encargos de importação.
4. Identificar a loja mais barata para cada produto e alertar os utilizadores cujo preço-alvo foi atingido.
5. Guardar o histórico numa base de dados, sem nunca eliminar dados.
6. Exportar os resultados para um servidor remoto e registar cada execução em *logs*.
7. Orquestrar o processo num *job* com tratamento de erros e execução automática.
8. Disponibilizar a informação num *dashboard* que permita acompanhar a evolução dos preços de cada produto.

1.3 Estrutura do documento

Além desta introdução, o documento está organizado em sete capítulos:

- **Capítulo 2 – Problema:** descreve o problema a resolver e os dados de teste, incluindo os erros introduzidos de propósito.
- **Capítulo 3 – Estratégia utilizada:** apresenta os requisitos, a arquitetura, as fontes de dados, as tecnologias escolhidas e os princípios de desenho.
- **Capítulo 4 – Transformações:** explica as transformações realizadas no KNIME, da uniformização das fontes ao cálculo do custo total e à seleção da melhor loja.
- **Capítulo 5 – Jobs:** descreve a orquestração do processo de carregamento na base de dados, alertas por e-mail, exportação, tratamento de erros, logs, execução em batch, agendamento e os testes realizados.
- **Capítulo 6 – Vídeo com demonstração:** indica o vídeo que demonstra o sistema em funcionamento.
- **Capítulo 7 – Conclusão e trabalhos futuros:** resume os resultados, as limitações e as melhorias possíveis.
- **Capítulo 8 – Referências:** lista as fontes consultadas.

2. Problema

O preço apresentado numa loja online nem sempre corresponde ao custo final suportado pelo consumidor. Nas compras internacionais, o valor total pode incluir portes de envio, conversão cambial, direitos aduaneiros, IVA de importação e encargos de desalfandegamento. Por esse motivo, comparar apenas os preços de etiqueta pode conduzir a decisões de compra menos vantajosas.

Este projeto pretende desenvolver um sistema de monitorização e comparação de preços que calcule o custo total de cada produto em quatro lojas internacionais e notifique o utilizador por e-mail quando o preço total atingir ou ficar abaixo do preço-alvo definido. O sistema deverá ainda manter um histórico de preços, evitar o envio repetido de alertas para a mesma condição, exportar ficheiros para um servidor remoto e disponibilizar um dashboard para acompanhar a evolução dos preços.

O cenário de funcionamento é simulado, recorrendo a diferentes formatos de dados e mecanismos de integração:

- Fontes de dados: quatro lojas, representadas por diferentes formatos de integração — Portugal (CSV), Espanha (JSON), Reino Unido (XML) e Estados Unidos (fluxo de dados em tempo real através do Node-RED).
- Normalização e validação: tratamento de moedas e formatos de preços distintos, identificação de produtos duplicados, validação de códigos EAN, deteção de preços em falta ou negativos e validação de endereços de e-mail.
- Cálculo do custo total: aplicação de regras de conversão cambial, portes de envio e encargos de importação, de acordo com as condições definidas para o cenário simulado.
- Preços-alvo: leitura e tratamento de preços definidos pelos utilizadores através de ficheiros JSON e CSV, incluindo formatos inconsistentes.
- Resultados esperados: registo persistente do histórico de preços, envio controlado de alertas por e-mail, exportação de ficheiros para um servidor remoto e visualização dos dados num dashboard.

O principal desafio consiste em integrar fontes de dados heterogéneas, garantir a consistência dos preços e calcular corretamente o custo total de cada produto, permitindo uma comparação fiável entre as lojas e uma monitorização contínua das condições definidas pelo utilizador.

3. Estratégia utilizada

A estratégia adotada assenta na utilização do KNIME como plataforma central de integração, transformação e processamento de dados. O workflow recolhe informação proveniente de diferentes fontes, aplica regras de validação e normalização, calcula o custo total dos produtos e encaminha os resultados para os serviços de persistência, notificação, exportação e visualização.

A arquitetura integra fontes de dados em diferentes formatos, nomeadamente CSV, JSON, XML e um fluxo de dados em tempo real disponibilizado através do Node-RED. Os serviços de apoio são executados em contentores Docker, permitindo isolar componentes e facilitar a configuração e a reprodução do ambiente de execução.

O processo é complementado por mecanismos de validação, registo de erros, preservação do histórico e monitorização das execuções. Os requisitos seguintes especificam as funcionalidades disponibilizadas pelo sistema e as propriedades de qualidade que orientam a sua implementação.

3.1 Requisitos

Os requisitos funcionais (Tabela 1) descrevem o que o sistema faz e os não funcionais (Tabela 2) descrevem as qualidades que deve ter.

Tabela 1Requisitos funcionais.

ID	Requisito funcional
RF01	Recolher preços de quatro lojas através de ficheiros CSV, JSON, XML e um fluxo de dados em tempo real.
RF02	Ler e interpretar a configuração do sistema a partir de um ficheiro YAML.
RF03	Obter o catálogo de produtos, as taxas de câmbio e as taxas de IVA através de APIs REST, quando aplicável.
RF04	Normalizar preços e portes de envio apresentados em diferentes formatos textuais e monetários.
RF05	Validar os códigos EAN-13, os endereços de e-mail e os preços, identificando e justificando os registos inválidos.
RF06	Identificar e excluir registos duplicados do processamento, preservando os dados brutos para auditoria.
RF07	Calcular o custo total dos produtos em euros, considerando as taxas de câmbio, os portes de envio e as regras de importação definidas.
RF08	Identificar a loja com menor custo total por produto e calcular a poupança relativamente à loja portuguesa.
RF09	Enviar alertas por e-mail quando o custo total for inferior ou igual ao preço-alvo, evitando alertas repetidos para a mesma condição durante 24 horas.
RF10	Armazenar o histórico de preços e os dados de referência numa base de dados.
RF11	Exportar os resultados em CSV e JSON e transferir os ficheiros para um servidor SFTP.
RF12	Registar cada execução, incluindo a data, o estado e os erros ocorridos.
RF13	Disponibilizar um dashboard para consultar a evolução dos preços e o estado do sistema.
RF14	Executar automaticamente o processo completo de hora a hora.

Tabela 2 Requisitos não funcionais

ID	Requisito não funcional
RNF01	Preservação dos dados: conservar os dados brutos, o histórico e os registos de auditoria, sem eliminação não autorizada.
RNF02	Integridade: garantir a consistência dos dados através de chaves estrangeiras, restrições e regras de domínio adequadas.
RNF03	Rastreabilidade: manter os dados de origem, os registos rejeitados, os motivos de rejeição e os logs das execuções.
RNF04	Robustez: registar falhas de serviços externos e permitir que o processamento continue quando estas não comprometam as etapas seguintes.
RNF05	Portabilidade: facilitar a execução noutras máquinas através de contentores Docker, caminhos configuráveis e scripts de arranque.
RNF06	Manutenibilidade: organizar e documentar o workflow por fases e manter o código e a configuração sob controlo de versões através do Git.

3.2 Arquitetura

A Figura 1apresenta a arquitetura do sistema, evidenciando a ligação entre as fontes de dados, o KNIME e os serviços de destino executados em contentores Docker.

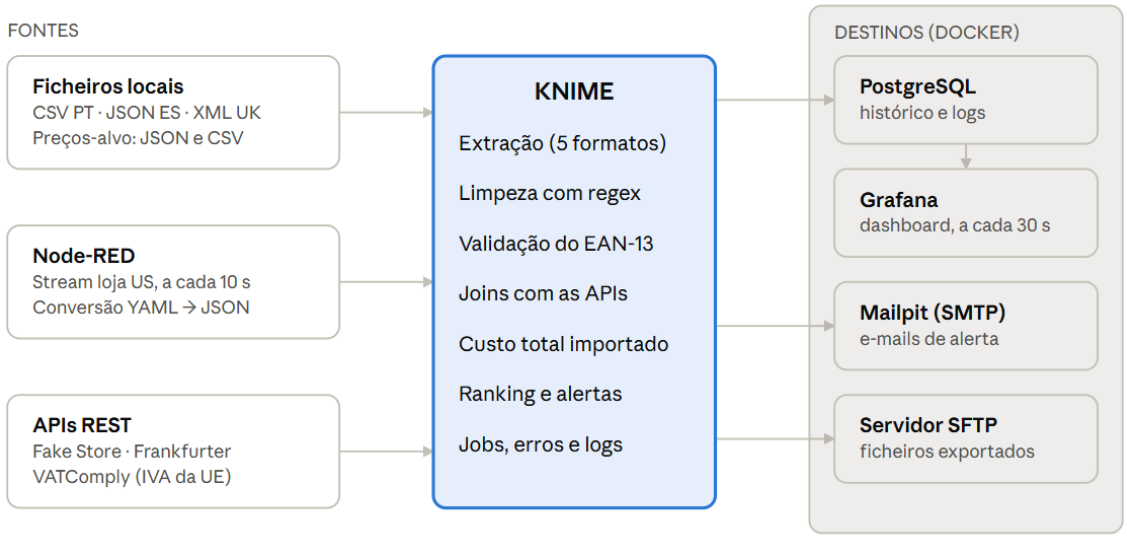


Figura 1 Arquitetura do sistema. Fonte: elaboração própria.

O KNIME constitui o núcleo do processamento, sendo responsável pela extração, limpeza, normalização e validação dos dados, pela integração das informações provenientes das APIs REST e pelo cálculo do custo total dos produtos. Posteriormente, determina as lojas mais económicas, gere os alertas e regista os resultados e as ocorrências de cada execução.

O Node-RED complementa o KNIME em três funções: simula o *stream* de preços da loja dos Estados Unidos e disponibiliza-o através de um *endpoint* HTTP, converte o ficheiro de configuração de YAML para JSON e agenda a execução automática do *workflow* de hora a hora.

O PostgreSQL armazena o histórico de preços e os logs do sistema. O Grafana consulta os dados armazenados no PostgreSQL para apresentar o dashboard e acompanhar a evolução dos preços. O Mailpit permite testar o envio e consultar os e-mails de alerta, enquanto o servidor SFTP recebe os ficheiros exportados pelo workflow.

O PostgreSQL armazena o histórico de preços, os dados de referência (lojas, produtos, taxas de câmbio e de IVA), os preços-alvo, os alertas enviados, os dados brutos, os registos rejeitados e os *logs* de execução. O Grafana consulta estes dados para apresentar o *dashboard* e acompanhar a evolução dos preços. O Mailpit recebe, por SMTP, os e-mails de alerta, que podem ser consultados numa página web sem serem enviados para a Internet, enquanto o servidor SFTP recebe os ficheiros exportados pelo *workflow*.

A arquitetura separa, assim, a recolha e o processamento de dados da sua persistência, visualização e distribuição, facilitando a manutenção e a monitorização do sistema.

3.3 Fontes de dados

O sistema integra nove fontes de dados, de três naturezas distintas: ficheiros locais, um fluxo de dados simulado em tempo real e APIs REST públicas. A Tabela 3 identifica cada fonte, o respetivo formato, a origem e o conteúdo disponibilizado.

Tabela 3 Fontes de dados, formatos e origem.

Fonte	***Formato***	***Origem***	***Conteúdo***
Loja PT	*CSV*	*ficheiro local*	*preços e portes em €, com erros*
Loja ES	*JSON*	*ficheiro local*	*preços "x,xx EUR", portes "Gratis" ou valor*
Loja UK	*XML*	*ficheiro local*	*preços e portes em £*
Loja US	*JSON (stream)*	*Node-RED, endpoint HTTP*	*preços em $ que variam a cada 10 s, com promoções relâmpago*
Configuração	*YAML → JSON*	*Node-RED converte o YAML*	*país de destino, lojas e regras de importação*
Preços-alvo	*JSON + CSV*	*ficheiros locais*	*e-mail, EAN e preço-alvo de cada utilizador*
Fake Store API	*JSON (REST)*	*API pública*	*catálogo e categoria dos produtos*
Frankfurter	*JSON (REST)*	*API pública*	*taxas de câmbio do BCE (GBP, USD)*
VATComply	*JSON (REST)*	*API pública*	*IVA normal por país e pertença à UE*

As quatro lojas foram desenhadas para representar a heterogeneidade encontrada em cenários reais. Cada loja usa um formato de ficheiro, uma moeda e uma convenção diferentes para escrever preços e portes. A loja portuguesa disponibiliza um ficheiro CSV separado por ponto e vírgula, com preços no formato "1.049,99 €". A loja espanhola usa JSON, com preços em "x,xx EUR" e portes indicados como "Gratis" ou com um valor. A loja do Reino Unido usa XML, com o código EAN guardado como atributo de cada produto e os valores em libras. A loja dos Estados Unidos é a única fonte dinâmica, o Node-RED gera, a cada 10 segundos, novos preços

em dólares, no formato americano “$1,049.99”, com variações de ±3 % e promoções relâmpago ocasionais, e disponibiliza-os ao KNIME através de um *endpoint* HTTP.

Nos ficheiros das lojas foram introduzidos erros de propósito, para testar a fase de transformação: um produto duplicado, um código EAN com dígito de controlo errado, um código EAN com a letra “O” no lugar de um zero, um preço em falta e um preço negativo.

Duas fontes descrevem o funcionamento do próprio sistema. O ficheiro de configuração, em YAML, define o país de destino, as lojas e as regras de importação, o Node-RED converte-o para JSON, o formato lido pelo KNIME.

Os preços-alvo dos utilizadores chegam em dois ficheiros, um JSON e um CSV, que também misturam convenções (“105,00 €”, “1.000,00”, “65.00”) e incluem um endereço de e-mail e um código EAN inválidos.

Por fim, três APIs REST públicas enriquecem os dados com informação real e atualizada. A Fake Store API fornece o catálogo e a categoria de cada produto. A Frankfurter fornece as taxas de câmbio de referência do Banco Central Europeu (BCE) para a libra e o dólar. A VATComply fornece a taxa normal de IVA de cada país e indica se pertence à UE, o que permite ao sistema decidir quando aplicar as regras de importação.

A combinação destas fontes permite testar a integração de dados heterogéneos, a normalização de diferentes formatos monetários, a validação dos registos e o enriquecimento da informação através de serviços externos. São utilizados CSV, JSON, XML e YAML, bem como dados recebidos através de um endpoint HTTP. Esta diversidade permite avaliar a capacidade do sistema para processar entradas com estruturas e níveis de qualidade distintos.

3.4 Tecnologias e justificação

A escolha das tecnologias seguiu quatro critérios: serem gratuitas ou de código aberto sempre que possível, corresponderem às ferramentas sugeridas no enunciado, serem adequadas às funções que desempenham e permitirem reproduzir o ambiente noutra máquina sem uma configuração manual complexa. A Tabela 4 apresenta as tecnologias selecionadas, a fase do processo em que atuam - Extração (E), Transformação (T) ou Carregamento (L, de Load) - o seu papel e a justificação da escolha.

Tabela 4 Tecnologias escolhidas e justificação.

Tecnologia	Fase	Papel no sistema	Justificação
KNIME Analytics Platform	E, T, L	Extração, transformação, carregamento e processamento dos dados	Plataforma ETL visual, sugerida no enunciado, com nós para leitura de ficheiros, regex, joins, bases de dados, e-mail, SFTP e tratamento de erros.
Node-RED	Ingestão e orquestração	Geração do stream dos EUA, endpoint HTTP e conversão YAML para JSON	Adequado à geração de dados contínuos, exposição de endpoints HTTP e integração de fluxos orientados a eventos.
PostgreSQL	L	Histórico, dados de referência, alertas, registos rejeitados e logs	Base de dados relacional que suporta restrições de integridade, vistas e triggers, permitindo aplicar regras de validação e proteção dos dados.
Grafana	Visualização	Dashboard de preços e estado do sistema	Permite consultar dados do PostgreSQL através de SQL e apresentar gráficos com atualização periódica.
Mailpit	Notificações de teste	Receção e consulta de e-mails de alerta	Permite testar o envio de mensagens por SMTP sem as entregar a destinatários reais.
Servidor SFTP (atmoz/sftp)	Exportação	Receção dos ficheiros exportados	Disponibiliza um servidor SFTP num contentor Docker, facilitando os testes de transferência de ficheiros.
Docker Compose	Transversal	Configuração e gestão dos serviços	Permite configurar e iniciar os serviços de apoio de forma coordenada e reproduzir o ambiente noutra máquina.
NodePit Batch	Execução	Execução do workflow KNIME sem interface gráfica	Permite executar workflows em modo batch e integrar a execução com um mecanismo externo de agendamento.
Fake Store API, Frankfurter, VATComply	E	Obtenção do catálogo, câmbios do BCE e taxas de IVA	APIs REST públicas e gratuitas, sem chave de acesso, com dados reais
Git / GitHub	Transversal	Controlo de versões e gestão do projeto	Permite acompanhar alterações, preservar o histórico dos ficheiros e facilitar a gestão do projeto.

O KNIME Analytics Platform foi escolhido como ferramenta central por ser uma das plataformas ETL sugeridas no enunciado e permitir construir visualmente o processo de integração e transformação dos dados. Os seus nós permitem implementar a leitura de ficheiros, a normalização com expressões regulares, a integração através de joins, os cálculos, o acesso à base de dados, o envio de e-mails, a transferência por SFTP e o controlo do fluxo de execução. Desta forma, as principais etapas do processamento ficam organizadas num único workflow, facilitando a sua compreensão e manutenção.

O Node-RED complementa o KNIME nas tarefas orientadas a eventos. É utilizado para gerar o stream simulado da loja dos Estados Unidos, disponibilizar os dados através de um endpoint HTTP e converter o ficheiro de configuração YAML para JSON. A coordenação da execução periódica deve ser descrita de acordo com o mecanismo de agendamento efetivamente utilizado.

Para a persistência foi escolhido o PostgreSQL, uma base de dados relacional que suporta restrições de integridade, vistas e triggers. Estes mecanismos permitem aplicar regras de validação e impedir operações de eliminação abrangidas pelas restrições implementadas. O Grafana consulta os dados armazenados no PostgreSQL para apresentar a evolução dos preços e o estado do sistema num dashboard com atualização periódica.

O Mailpit e o servidor SFTP, executados em contentores, permitem testar os mecanismos de notificação e transferência de ficheiros utilizando os respetivos protocolos. O Mailpit recebe as mensagens de teste sem as entregar aos destinatários reais, enquanto o servidor SFTP recebe os ficheiros exportados. Esta configuração facilita a realização de testes num ambiente controlado, sem utilizar serviços externos de produção.

Por fim, o Docker Compose permite configurar e iniciar os serviços de apoio de forma coordenada, facilitando a reprodução do ambiente noutra máquina. O NodePit Batch é utilizado para executar o workflow sem a interface gráfica do KNIME, enquanto o Git, através do repositório no GitHub, permite controlar as versões e manter o histórico dos ficheiros do projeto. As APIs REST utilizadas complementam o processo com dados de catálogo, taxas de câmbio e informação fiscal.

3.5 Princípios de desenho

O sistema foi concebido segundo cinco princípios fundamentais, destinados a garantir a integridade, a rastreabilidade e a preservação dos dados, bem como a portabilidade da solução.

Preservação e auditoria dos dados. Triggers na BD recusam DELETE e TRUNCATE em todas as tabelas, recusam UPDATE nas tabelas de factos e guardam na tabela auditoria a versão antiga de cada linha alterada. Um event trigger recusa DROP TABLE e DROP SCHEMA.

Portabilidade dos caminhos. Os ficheiros são lidos e escritos através da Workflow Data Area do KNIME, utilizando caminhos relativos no formato knime://knime.workflow/data/.... Esta abordagem reduz a dependência de caminhos específicos de cada computador e facilita a execução do projeto noutros ambientes.

Preservação dos dados durante a limpeza. Os dados originais são conservados e os registos rejeitados durante a validação são armazenados com a respetiva justificação. Desta forma, é possível identificar os problemas encontrados e acompanhar o tratamento aplicado aos dados.

Validação dos preços. O KNIME rejeita preços nulos ou negativos, enquanto restrições CHECK na base de dados aplicam regras de integridade adicionais. Esta validação em duas camadas reduz o risco de dados inválidos chegarem às etapas seguintes do processamento.

Imutabilidade dos dados de entrada. Os ficheiros de origem são utilizados apenas para leitura, sem serem alterados pelo processo. Estes ficheiros são mantidos sob controlo de versões no Git, enquanto uma cópia dos dados brutos de cada execução é armazenada na tabela precos_brutos, permitindo preservar o conteúdo recebido para consulta e auditoria.

4. Transformações

Este capítulo descreve a fase de transformação, implementada num único *workflow* do KNIME (01_extracao). O *workflow* recebe os dados das quatro lojas, dos preços-alvo e das APIs, limpa-os e valida-os, cruza as diferentes fontes, calcula o custo total real de cada produto e determina a melhor loja e os alertas a enviar.

A (acedido em 15 de outubro de 2026).

Docker Inc. (s.d.). *Docker Compose overview*. https://docs.docker.com/compose/ (acedido em 15 de outubro de 2026).

Fake Store API. (s.d.). *Fake Store API*. https://fakestoreapi.com/ (15 de outubro de 2026).

Ferreira, L., & Ribeiro, Ó. (2026). *Integração de Sistemas de Informação – Trabalho Prático I: enunciado*. Licenciatura em Engenharia de Sistemas Informáticos.

Frankfurter. (s.d.). *Frankfurter: exchange rates API*. https://www.frankfurter.app/ (acedido em 15 de outubro de 2026).

Grafana Labs. (s.d.). *PostgreSQL data source*. https://grafana.com/docs/grafana/latest/datasources/postgres/ (acedido em 17 de outubro de 2026).

GS1. (s.d.). *How to calculate a check digit manually*. https://www.gs1.org/services/how-calculate-check-digit-manually (acedido em 16 de outubro de 2026).

Hitachi Vantara. (s.d.). *About Kitchen, Pan, and Carte*. Pentaho Documentation. https://docs.pentaho.com/pdia-try-pdia/about-pentaho-workflows/pentaho-data-integration-workflows/evaluate-and-learn-pentaho-data-integration-pdi/about-kitchen-pan-and-carte (acedido em 19 de outubro de 2026).

JSON Lines. (s.d.). *JSON Lines*. https://jsonlines.org/ (acedido em 17 de outubro de 2026).

KNIME AG. (s.d.). *KNIME Documentation*. https://docs.knime.com/ (acedido em 13 de outubro de 2026).

KNIME Forum. (2026, 13 de abril). *Batch execution no longer supported*. https://forum.knime.com/t/batch-execution-no-longer-supported/91773 (acedido em 18 de outubro de 2026).

Mailpit. (s.d.). *Mailpit documentation*. https://mailpit.axllent.org/ (acedido em 18 de outubro de 2026).

NodePit. (s.d.). *NodePit Batch*. https://nodepit.com/product/batch (acedido em 18 de outubro de 2026).

OpenJS Foundation. (s.d.). *Node-RED documentation*. https://nodered.org/docs/ (acedido em 18 de outubro de 2026).

Oracle. (s.d.). *Class Pattern* (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html (acedido em 18 de outubro de 2026).

PostgreSQL Global Development Group. (s.d.). *CREATE TRIGGER*. https://www.postgresql.org/docs/current/sql-createtrigger.html (17 de outubro de 2026).

PostgreSQL Global Development Group. (s.d.). *Event triggers*. https://www.postgresql.org/docs/current/event-triggers.html (17 de outubro de 2026).

VATComply. (s.d.). *VATComply API*. https://www.vatcomply.com/ apresenta a visão geral do *workflow*. Os nós estão organizados em zonas, da esquerda para a direita, que correspondem às etapas do processo: extração e uniformização das fontes, limpeza e validação, cruzamento de dados e cálculo do custo total, seleção da melhor loja e, por fim, os *jobs* de carregamento descritos no Capítulo 5. Cada nó tem um comentário com o número da etapa a que pertence (por exemplo, “5.2 Motivo rejeição”), o que facilita a leitura e a manutenção.

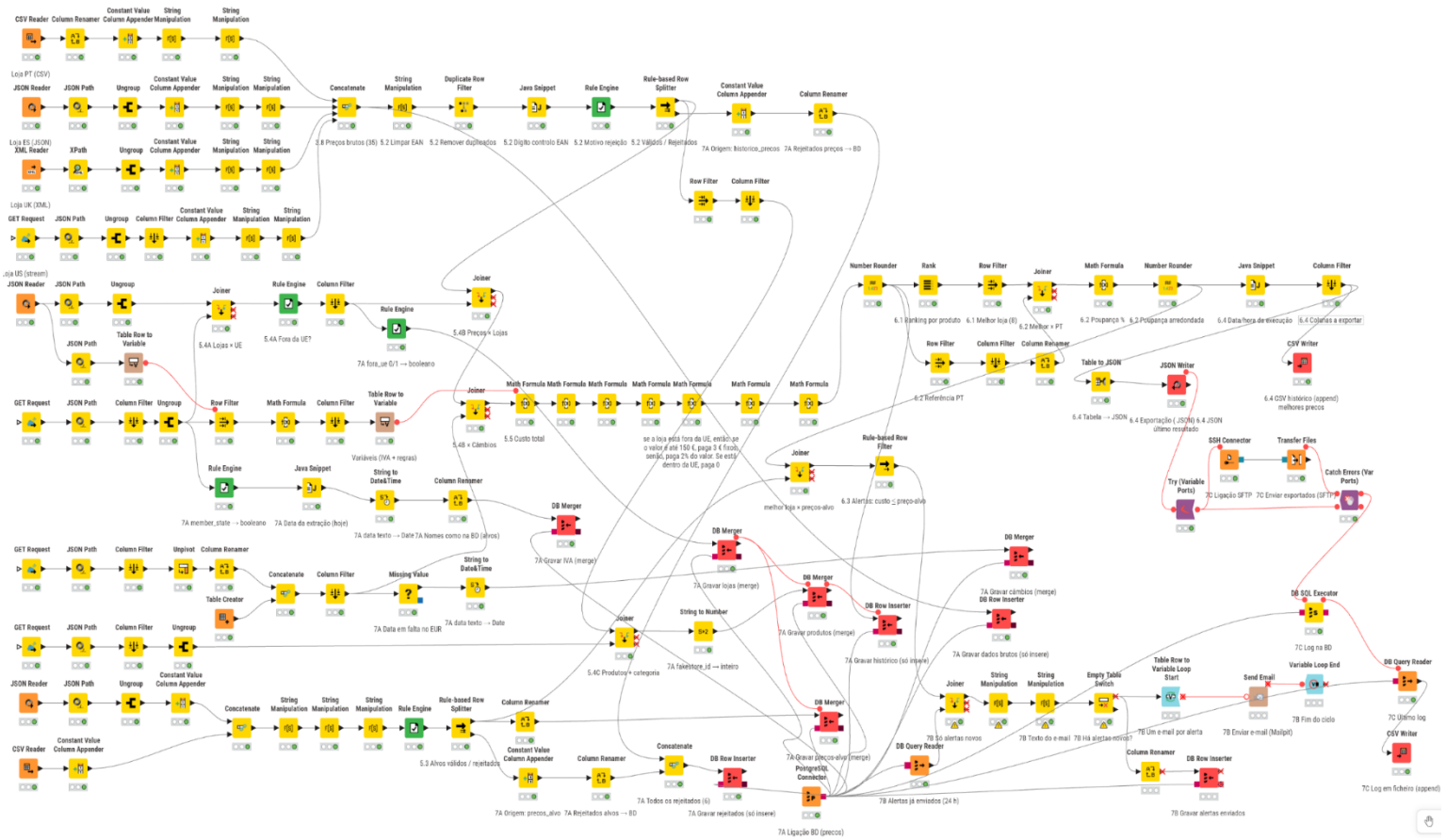


Figura 2 Visão geral do workflow 01_extracao, organizado por zonas.

A Tabela 5 resume a evolução do número de registos de preços ao longo da transformação, numa execução. Nenhum registo se perde sem deixar rasto. Os dados brutos e os rejeitados ficam guardados na base de dados, e só os duplicados exatos são descartados.

Tabela 5 Evolução do número de registos de preços numa execução.

Etapa	Variação de linhas	Resultado	Destino
Dados brutos recebidos	+35	35 linhas	precos_brutos
Remoção de duplicados	−1	34 linhas	Não aplicável
Validação dos dados	-4	30 linhas válidas	rejeitados, para as linhas rejeitadas
Cálculo do custo total	0	30 linhas processadas	historico_precos

As secções seguintes descrevem cada etapa pela ordem em que é executada: a uniformização das quatro lojas (4.1), a limpeza com expressões regulares (4.2), a validação (4.3), o enriquecimento com *joins* (4.4), o cálculo do custo total (4.5) e a seleção da melhor loja e dos alertas (4.6).

4.1 Uniformização das quatro lojas

O objetivo desta etapa é transformar quatro fontes com formatos e estruturas diferentes numa única tabela com um esquema comum. Cada loja utiliza o seu próprio formato de ficheiro e diferentes nomes de campos para representar a mesma informação, como se apresenta na Tabela 6.

Tabela 6 Mapeamento dos campos de cada loja para o esquema comum.

Campo comum	**Loja PT (CSV)**	**Loja ES (JSON)**	**Loja UK (XML)**	**Loja US (*stream*)**
ean	ean	codigo_ean	@ean (atributo XML)	ean
titulo	titulo	nombre	name	name
preco_texto	preco	precio	price	price
portes_texto	portes	envio	shipping	shipping

Cada loja é processada numa linha própria do workflow, utilizando os nós adequados ao respetivo formato, conforme ilustrado na Figura 3:

- Loja PT (CSV): o nó CSV Reader lê o ficheiro, utilizando o ponto e vírgula como separador.
- Loja ES (JSON): a sequência JSON Reader → JSON Path → Ungroup permite extrair os produtos da coleção productos e criar uma linha por produto.
- Loja UK (XML): a sequência XML Reader → XPath → Ungroup extrai os campos /shop/product/@ean, name, price e shipping, produzindo uma linha por produto.
- Loja US (stream): o nó GET Request consulta o endpoint http://localhost:1880/loja-us/precos, disponibilizado pelo Node-RED. O nó JSON Path extrai do corpo da resposta (body) os campos ean, name, price e shipping, através das expressões $[*].ean, $[*].name, $[*].price e $[*].shipping. De seguida, o nó Ungroup organiza os produtos em linhas individuais e o Column Filter remove as colunas técnicas Status e Content type, acrescentadas à resposta HTTP, que não fazem parte dos dados dos produtos.

A atribuição dos nomes do esquema comum é realizada de duas formas. Na loja PT, utiliza-se o nó Column Renamer, uma vez que o CSV Reader mantém os cabeçalhos originais do ficheiro. Nas lojas ES, UK e US, os nomes são definidos diretamente na configuração dos nós JSON Path e XPath, de modo que as colunas de saída já correspondam ao esquema comum.

Em todas as linhas, o nó Constant Value Column Appender acrescenta duas colunas de rastreabilidade: loja, que identifica a loja de origem (PT, ES, UK ou US), e origem, que identifica o formato ou mecanismo de obtenção dos dados (csv, json, xml ou stream). Nesta fase são também aplicadas operações iniciais de limpeza aos campos de preço e portes, recorrendo a nós String Manipulation e a expressões regulares. O detalhe destas operações é apresentado na secção 4.2, uma vez que as fontes utilizam convenções numéricas diferentes, nomeadamente a vírgula decimal nas lojas PT e ES e o ponto decimal nas lojas UK e US.

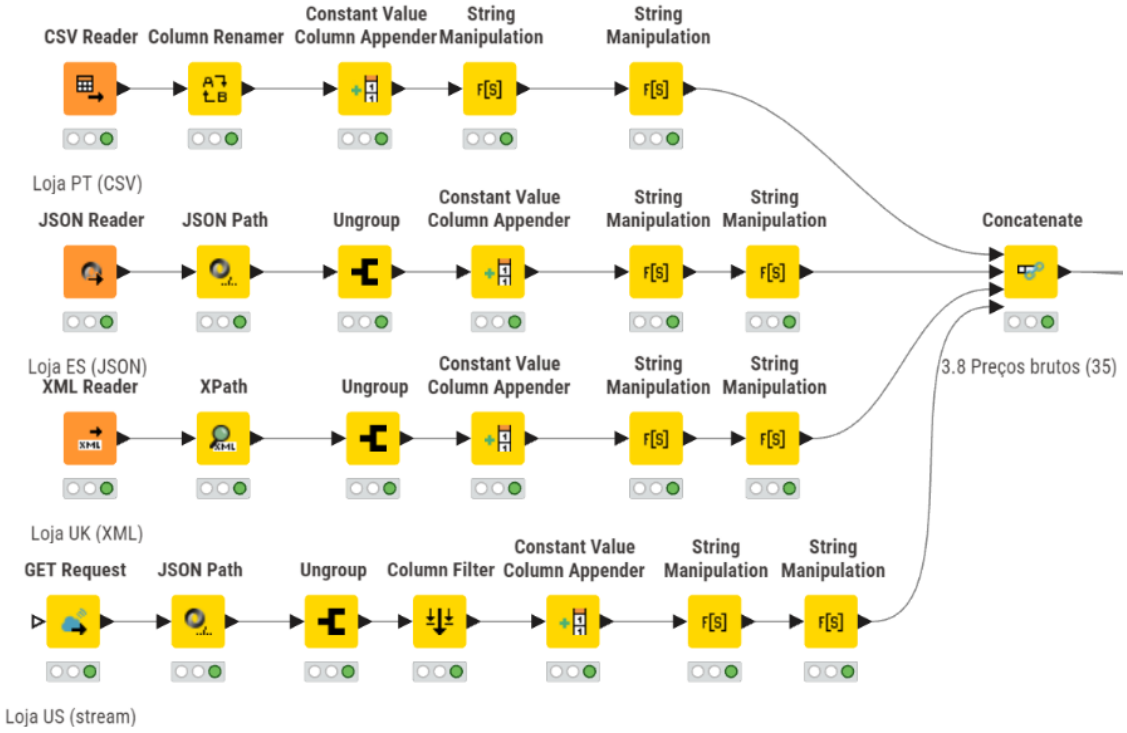


Figura 3 Extração e uniformização das quatro lojas.

Por fim, o nó Concatenate, configurado no modo Union, combina as quatro linhas de processamento numa única tabela. O resultado contém 35 registos e as colunas ean, titulo, preco_texto, portes_texto, loja e origem. Antes da validação, estes registos são armazenados na tabela precos_brutos, permitindo preservar os dados recebidos após a extração e a uniformização inicial.

4.2 Limpeza com expressões regulares

Após a uniformização das fontes, os preços e os portes continuam representados como texto, seguindo convenções diferentes: vírgula decimal e ponto de milhares nas lojas PT e ES (por exemplo, 1.049,99 €), e ponto decimal e vírgula de milhares nas lojas UK e US (por exemplo, $1,049.99). Os preços-alvo dos clientes podem também apresentar formatos distintos. Para transformar estes valores em números comparáveis, são utilizadas expressões regulares (regular expressions) no nó String Manipulation, através da função regexReplace, que utiliza a sintaxe de expressões regulares do Java. Em cada linha do workflow, um nó trata o preço e outro trata os portes. A Tabela 7 apresenta as principais expressões utilizadas nesta etapa.

Tabela 7 Funções e expressões utilizadas na limpeza dos preços, portes e endereços de e-mail.

Caso	Exemplo	Expressão ou operação
Preço PT/ES	1.049,99 € → 1049.99	toDouble(replace(regexReplace($preco_texto$, "[^0-9,]", ""), ",", "."))
Portes PT/ES gratuitos	Grátis → 0	regexReplace($portes_texto$, "(?iu)gr[aá]tis|free", "0"), antes da limpeza numérica
Preço UK/US	$1,000.37 → 1000.37	toDouble(regexReplace($preco_texto$, "[^0-9.]", ""))
Preço-alvo	1.000,00 → 1000	Remoção do ponto de milhares com [.](?=[0-9]{3}([^0-9]|$)), seguida da substituição da vírgula por ponto
E-mail	ANA.Silva@exemplo.pt → ana.silva@exemplo.pt	lowerCase(strip($email$))

As expressões utilizadas para limpar os preços recorrem a classes de caracteres negadas. No caso de PT e ES, [^0-9,] corresponde a qualquer carácter que não seja um algarismo nem uma vírgula. Esta operação remove símbolos monetários, espaços e o ponto utilizado como separador de milhares. Em seguida, a vírgula é substituída por um ponto e a função toDouble converte o resultado numérico para um tipo de dados numérico. Nas lojas UK e US, a expressão [^0-9.] preserva o ponto decimal e remove os restantes caracteres não numéricos.

Duas expressões merecem uma explicação adicional:

- (?iu)gr[aá]tis|free reconhece os portes gratuitos escritos de várias formas (“Grátis”, “Gratis”, “GRATIS”, “Free”). A opção i ignora a diferença entre maiúsculas e minúsculas, a opção u estende essa regra às letras acentuadas e a classe [aá] aceita a palavra com ou sem acento.
- [.](?=[0-9]{3}([^0-9]|$)) remove o ponto dos milhares sem afetar o ponto decimal. O lookahead (?=…) só aceita um ponto seguido de exatamente três dígitos e, depois deles, de um carácter não numérico ou do fim do texto. Assim, em “1.000,00” o ponto é removido, por ser separador de milhares, mas em “65.00” é mantido, por ser decimal. Desta forma, os dois formatos presentes nos preços-alvo são tratados pela mesma expressão.

Importa ainda considerar que as expressões de limpeza dos preços removem o sinal negativo, uma vez que este não pertence às classes de caracteres preservadas. Por exemplo, -15,00 € poderia ser convertido em 15.00. Por esse motivo, a validação dos valores negativos deve considerar o texto original antes desta conversão, caso contrário, o sinal pode perder-se e o valor inválido passar despercebido. O tratamento efetivamente implementado é descrito na secção 4.3.

A Figura 4 apresenta o resultado da limpeza dos preços e dos portes da loja UK. As colunas *preco_texto* e *portes_texto* conservam os valores textuais de origem, enquanto *preco_original* e *portes_original* apresentam os respetivos valores numéricos após a conversão. Nesta fase, os valores permanecem expressos na moeda de origem; a conversão para euros e o cálculo do custo total são tratados nas secções 4.4 e 4.5.

Rows: 7 | Columns: 8 Sorting Filter

#	RowID	ean String	titulo String	preco_texto String	portes_texto String	loja String	origem String	preco_original Number (Float)	portes_original Number (Float)
1	Row0_1	5601234000016	Fjallraven Foldsack Backpack 15in	£94.56	£12.00	UK	xml	94.56	12
2	Row0_2	5601234000030	WD Elements 2TB Portable HDD	£53.12	£12.00	UK	xml	53.12	12
3	Row0_3	5601234000047	SanDisk SSD Plus 1TB	£91.56	£12.00	UK	xml	91.56	12
4	Row0_4	5601234000054	Silicon Power 256GB SSD	£27.05	£12.00	UK	xml	27.05	12
5	Row0_5	5601234000061	WD 4TB Gaming Drive PS4	£96.90	£12.00	UK	xml	96.9	12
6	Row0_6	5601234000078	Acer SB220Q 21.5in Monitor	£95.92	£12.00	UK	xml	95.92	12
7	Row0_7	5601234000085	Samsung 49in Curved CHG90	£829.49	£29.99	UK	xml	829.49	29.99

Figura 4 Preços e portes da loja UK antes e depois da limpeza.

4.3 Validação

Antes do cálculo, cada linha é verificada segundo três critérios: o preço tem de ser positivo, os portes não podem ser negativos (os portes gratuitos, com valor 0, são válidos) e o código EAN-13 tem de ser válido. Primeiro, o nó Duplicate Row Filter remove os registos repetidos, mantendo apenas uma ocorrência de cada combinação de loja e EAN; nesta execução foi removida uma linha.

O EAN-13 termina num dígito de controlo, calculado a partir dos 12 dígitos anteriores. Cada dígito d_i é multiplicado por um peso w_i, igual a 1 nas posições ímpares e a 3 nas posições pares, e o dígito de controlo é dado por (Figura 5 Cálculo do EAN):

$$d_{13} = \left(10 - \left(\sum_{i=1}^{12} d_i \cdot w_i\right) \bmod 10\right) \bmod 10, \quad w_i \in \{1, 3\}$$

Figura 5 Cálculo do EAN

O Excerto 1 mostra o código do nó Java Snippet que aplica esta regra. Um código só é aceite se tiver 13 dígitos e se o último coincidir com o dígito calculado; o resultado fica na coluna ean_digito_ok (1 = válido, 0 = inválido).

```
// Excerto 1 – Validação do dígito de controlo do EAN-13 (nó Java
Snippet)
String e = c_ean;
if (e == null || !e.matches("[0-9]{13}")) {
    out_ean_digito_ok = 0;                        // formato inválido
} else {
    int soma = 0;
    for (int i = 0; i < 12; i++) {
        int d = e.charAt(i) - '0';
        soma += (i % 2 == 0) ? d : d * 3;         // pesos 1, 3, 1, 3,
...
    }
    out_ean_digito_ok = ((10 - soma % 10) % 10 == e.charAt(12) - '0') ?
1 : 0;
}
```

Em seguida, um nó Rule Engine atribui a cada linha um motivo de rejeição, e um nó Rule-based Row Splitter separa as linhas sem motivo, que seguem para o cálculo, das restantes, que são gravadas na tabela rejeitados juntamente com o motivo. As regras são avaliadas por ordem e a primeira que se verifica define o motivo (Excerto 2).

```
// Excerto 2 – Regras do nó Rule Engine que atribuem o motivo de rejeição
MISSING $preco_original$          => "preco em falta"
MISSING $portes_original$         => "portes em falta"
$preco_texto$ LIKE "*-*"          => "preço negativo"
$portes_texto$ LIKE "*-*"         => "portes negativos"
$preco_original$ <= 0             => "preço igual a zero"
NOT $ean$ MATCHES "[0-9]{13}"     => "EAN com formato invalido"
$ean_digito_ok$ = 0             => "EAN com digito de controlo errado"
TRUE                              => ""
```

Os preços negativos são detetados no texto original, com a regra $preco_texto$ LIKE "*-*", porque a limpeza com expressões regulares remove o sinal (secção 4.2). Uma regra adicional rejeita os preços iguais a zero. Como segunda barreira, a base de dados (BD) tem regras CHECK que recusam qualquer preço não positivo que escape ao KNIME: chk_precos_positivos nos preços e chk_alvo_positivo nos preços-alvo. Num teste, um INSERT com o preço −10 € foi recusado com a mensagem violates check constraint "chk_precos_positivos".

A Figura 6 mostra as linhas rejeitadas à saída do Rule-based Row Splitter. Dois casos confirmam as decisões descritas acima. Na Pen USB, o valor limpo (preco_original) é 15, sem sinal, o que demonstra que a limpeza remove o "–" e que a deteção só é possível no texto original. No Cable HDMI, o EAN falha também o dígito de controlo (ean_digito_ok = 0), mas o motivo gravado é "EAN com formato invalido", porque essa regra aparece primeiro e o Rule Engine regista apenas a primeira regra que se verifica.

RowID	ean String	titulo String	...	preco_texto String	portes_texto String	stock Str...	fak... Str...	loja Str...	origem String	preco_original Number (Float)	portes_original Number (Float)	ean_digito_ok Number (Integer)	motivo_rejeicao String
Row9	5601234000999	Rato Logitech M185		14,99€	3,99 €	Sim	?	PT	csv	14.99	3.99	0	EAN com digito de controlo errado
Row10	5601234000092	Teclado Genérico		?	3,99 €	Sim	?	PT	csv	?	3.99	1	preco em falta
Row11	5601234000108	Pen USB SanDisk 64GB		-15,00 €	Grátis	Sim	?	PT	csv	15	0	1	preço negativo
Row19	5601234000035	Cable HDMI 2m		9,99 EUR	6,95 EUR	?	?	ES	json	9.99	6.95	0	EAN com formato invalido

Figura 6 Linhas rejeitadas pela validação, com o respetivo motivo.

Os preços-alvo dos clientes chegam de dois ficheiros, precos_alvo.json e precos_alvo.csv, e passam por uma validação própria, que verifica o formato do e-mail e do EAN. Dos 8 preços-alvo, 6 são válidos e 2 são rejeitados: um por ter um e-mail sem domínio de topo (rui.costa@exemplo) e outro por ter um EAN com apenas 5 dígitos. A Tabela 8 resume todos os registos rejeitados numa execução.

Tabela 8 Registos rejeitados numa execução e respetivo motivo.

Origem	Registo rejeitado	Motivo gravado
Preços	Rato Logitech M185 (PT), EAN 5601234000999	EAN com digito de controlo errado
Preços	Teclado Genérico (PT), sem preço	preco em falta
Preços	Pen USB SanDisk 64GB (PT), preço −15,00 €	preço negativo
Preços	Cable HDMI 2m (ES), EAN 56O1234000035 (letra O)	EAN com formato invalido
Preços-alvo	e-mail rui.costa@exemplo (JSON)	email invalido
Preços-alvo	EAN 12345 (CSV)	EAN invalido

4.4 Enriquecimento com joins

Após a validação, cada registo de preço contém o valor numérico do produto e dos portes na moeda da respetiva loja. Para calcular o custo total de uma compra a partir de Portugal, é necessário associar-lhe informação complementar, nomeadamente a moeda, a taxa de câmbio, a pertença do país da loja à União Europeia (UE), a taxa de IVA de destino e as regras de importação consideradas no cenário. Este enriquecimento é realizado através de nós Joiner, que combinam tabelas com base em colunas-chave, e de variáveis de fluxo (flow variables), utilizadas para disponibilizar parâmetros comuns às etapas seguintes. A Figura 7 apresenta a zona correspondente do workflow, e a Tabela 9 resume as principais junções realizadas.

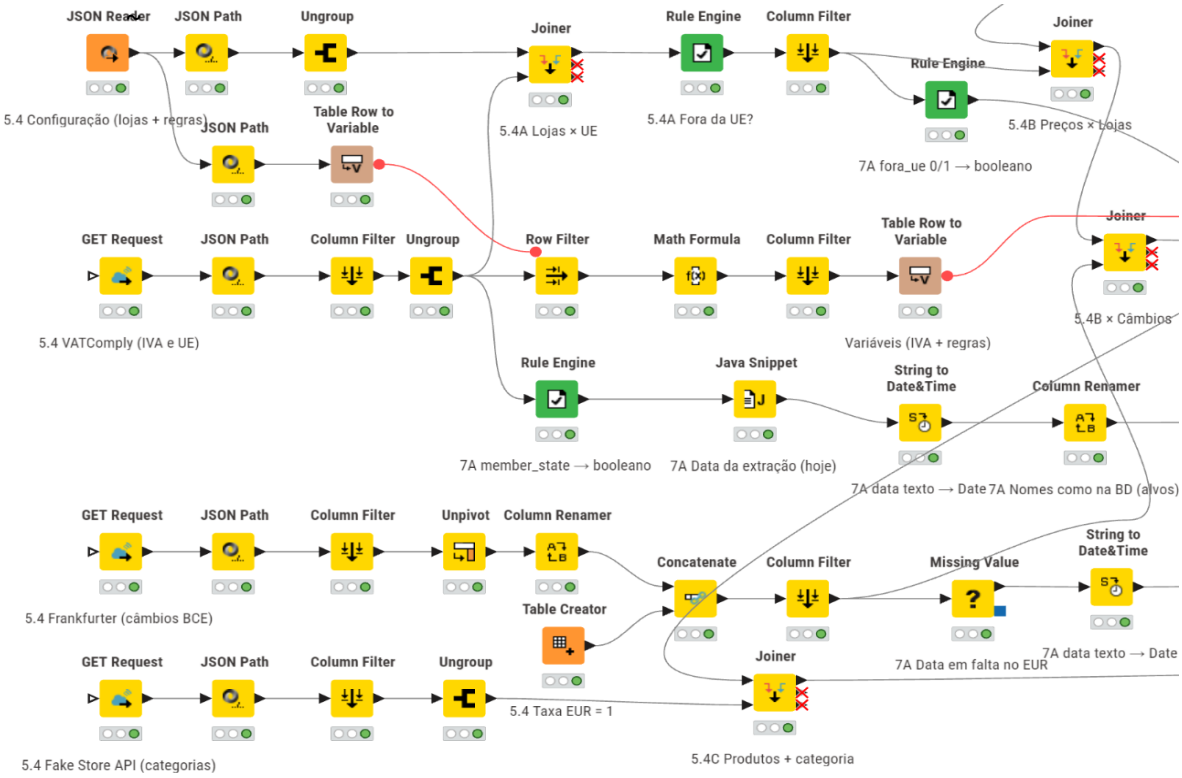


Figura 7 Enriquecimento: obtenção das tabelas de referência (configuração, VATComply, Frankfurter e Fake Store API) e junções com os preços.

Tabela 9 Junções realizadas no enriquecimento dos preços.

Tabela da esquerda	Tabela da direita	Chave	Tipo de *join*	Colunas acrescentadas
Lojas (YAML)	Países da UE (VATComply)	pais	*Left outer*	marcação da UE → fora_ue
Preços	Lojas	loja	*[Inner]*	moeda, pais, fora_ue
Preços + lojas	Câmbios (Frankfurter/BCE)	moeda	*[Inner / Left outer]*	taxa
Produtos	Catálogo (Fake Store API)	fakestore_id	*[Left outer]*	categoria

A primeira junção (5.4A) associa as lojas lidas do ficheiro de configuração à lista de países obtida através da API VATComply. Foi usado um *left outer join* de forma intencional, com um *inner join*, as lojas cujo país não pertence à UE (Reino Unido e Estados Unidos) seriam eliminadas, quando é precisamente a ausência de correspondência que as identifica. Um nó Rule Engine atribui então fora_ue_n = 1 a essas lojas e fora_ue_n = 0 às restantes. A Figura 8 mostra o resultado das lojas PT e ES que encontram correspondência, com IVA de 23 % e 21 % e member_state verdadeiro, enquanto as lojas UK (país GB) e US ficam com esses campos vazios e são marcadas como fora da UE.

Rows: 4 | Columns: 8 Sorting Filter

#	RowID	codigo String	nome String	pais String	moeda String	country_code String	standard_rate Number (Float)	member_state String	fora_ue_n Number (Integer)
1	Row0...	ES	TiendaES	ES	EUR	ES	21	true	0
2	Row0...	PT	LojaPT	PT	EUR	PT	23	true	0
3	Row0...	UK	ShopUK	GB	GBP	?	?	?	1
4	Row0...	US	StoreUS	US	USD	?	?	?	1

Figura 8 Resultado do left outer join entre as lojas e os países da VATComply.

As duas junções seguintes (5.4B) associam cada preço à respetiva loja, o que lhe acrescenta a moeda e a marcação fora_ue_n, e depois à taxa de câmbio dessa moeda. As taxas são obtidas da API Frankfurter, que publica as taxas de referência do Banco Central Europeu (BCE) para a libra (GBP) e o dólar (USD). Como a API não devolve a taxa do euro face a si próprio, um nó Table Creator acrescenta uma linha EUR com taxa 1, que se junta às restantes através de um Concatenate. Assim, os preços das lojas PT e ES atravessam as mesmas fórmulas sem conversão. Nestas junções usa-se um *inner join*, porque todos os preços válidos têm loja e moeda conhecidas, um preço sem correspondência indicaria um erro de configuração e não deve seguir para o cálculo.

A junção 5.4C associa os produtos ao catálogo da Fake Store API, através da coluna fakestore_id, para obter a categoria de cada produto. Neste caso usa-se um *left outer join*, porque nem todos os produtos têm correspondência no catálogo, sendo mantidos apenas sem categoria.

Por fim, os parâmetros comuns ao cálculo não são associados individualmente a cada preço através de junções. Em vez disso, são disponibilizados como variáveis de fluxo através do nó Table Row to Variable. A taxa de IVA de destino é obtida a partir dos dados da VATComply relativos a Portugal e convertida num fator decimal; os restantes parâmetros, incluindo o limite de baixo valor e os encargos definidos para o cenário, são lidos do ficheiro de configuração. A Tabela 10 resume as variáveis utilizadas nas fórmulas da secção 4.5.

Tabela 10 Variáveis de fluxo usadas no cálculo do custo total.

Variável	**Valor**	**Origem**	**Significado**
destino	PT	configuração	país de destino das compras
iva_destino	0,23	VATComply	taxa de IVA normal de Portugal
limite_baixo_valor_eur	150	configuração	limite das encomendas de baixo valor (€)

direito_fixo_baixo_valor_eur	3	configuração	direito fixo até ao limite (€)
direito_pct_acima_limite	0,02	configuração	direito acima do limite (2 % do valor)
taxa_desalfandegamento_eur	2	configuração	taxa de desalfandegamento (€)

4.5 Cálculo do custo total

Com os preços enriquecidos, o passo seguinte é calcular o custo real de cada produto entregue em Portugal, de forma a comparar lojas com moedas e regras fiscais diferentes. O cálculo é feito por sete nós Math Formula em sequência, cada um responsável por uma parcela, seguidos de um nó Number Rounder que arredonda os valores a duas casas decimais (Figura 9). As regras de importação não estão escritas nas fórmulas, estas são lidas das variáveis de fluxo apresentadas na Tabela 10, pelo que uma alteração no ficheiro de configuração se reflete no cálculo sem modificar o *workflow*.

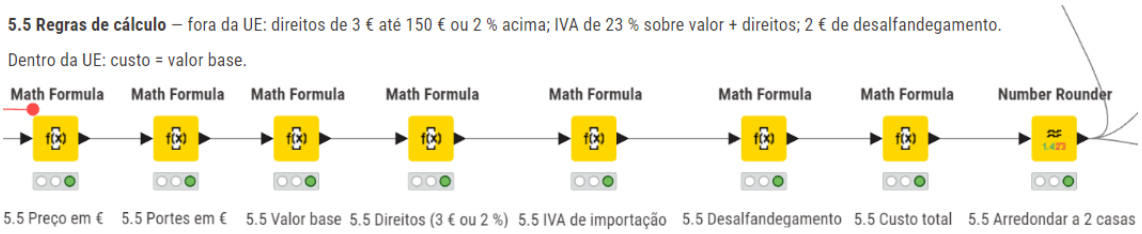


Figura 9 Cálculo do custo total com sete nós Math Formula e arredondamento final.

A Tabela 11 apresenta a expressão de cada nó, pela ordem em que são executados.

Tabela 11 Expressões dos nós Math Formula do cálculo do custo total.

Nó	Coluna criada	Expressão
1	preco_eur	$preco_original$ / $taxa$
2	portes_eur	$portes_original$ / $taxa$
3	valor_base	$preco_eur$ + $portes_eur$
4	direitos_eur	if($fora_ue_n$ == 1, if($valor_base$ <= $${Dlimite_baixo_valor_eur}$$, $${Ddireito_fixo_baixo_valor_eur}$$, $valor_base$ * $${Ddireito_pct_acima_limite}$$), 0)
5	iva_import_eur	if($fora_ue_n$ == 1, ($valor_base$ + $direitos_eur$) * $${Diva_destino}$$, 0)
6	desalfand_eur	if($fora_ue_n$ == 1, $${Dtaxa_desalfandegamento_eur}$$, 0)
7	custo_total_eur	$valor_base$ + $direitos_eur$ + iva_import_eur + $desalfand_eur$

Para uma loja fora da UE, com valor base v=preço/taxa + portes/taxa, o cálculo corresponde a (Figura 11 Cálculo dos direitos e a Figura 10 Cálculo do custo):

$$\text{direitos} = \begin{cases} 3\ € & v \le 150\ € \\ 0{,}02 \cdot v & v > 150\ € \end{cases}$$

Figura 11 Cálculo dos direitos

$$\text{custo} = v + \text{direitos} + 0{,}23 \cdot (v + \text{direitos}) + 2\ €$$

Figura 10 Cálculo do custo

Duas decisões merecem nota. Em primeiro lugar, o IVA de importação incide sobre o valor base acrescido dos direitos, e não apenas sobre o valor do produto, uma vez que os direitos aduaneiros fazem parte da base tributável na importação. Em segundo lugar, nas lojas da UE o custo total é igual ao valor base, porque se assume que o preço apresentado já inclui IVA, trata-se de uma simplificação, retomada nas limitações do capítulo 7. As três parcelas de importação (nós 4 a 6) usam a coluna fora_ue_n para valer 0 nas lojas da UE, o que permite tratar todas as lojas com as mesmas fórmulas.

A Figura 12 mostra o resultado para a Mochila Fjallraven nas quatro lojas e a Tabela 12 detalha o cálculo, verificado manualmente.

RowID	ean String	titulo String	loja String	preco_original Number (Float)	portes_original Number (Float)	valor_base Number (Float)	direitos_eur Number (Float)	iva_import_eur Number (Float)	desalfand_eur Number (Float)	custo_total_eur Number (Float)	taxa Nu...
Row0...	56012340000...	Mochila Fjallraven F...	PT	109.95	0	109.95	0	0	0	109.95	1
Row1...	56012340000...	Mochila Fjallraven F...	ES	106.65	0	106.65	0	0	0	106.65	1
Row2...	56012340000...	Fjallraven Foldsack ...	UK	94.56	12	127.12	3	29.93	2	162.05	0.838
Row2...	56012340000...	Fjallraven Foldsack ...	US	88.36	24.99	100.9	3	23.9	2	129.8	1.123

Figura 12 Custo total da Mochila Fjallraven nas quatro lojas.

Tabela 12 Cálculo do custo total da Mochila Fjallraven (execução de 22 de outubro de 2026).

Loja	Preço + portes	Taxa	Valor base	Direitos	IVA importação	Desalfand.	Custo total
PT	109,95 € + 0	1	109,95 €	0	0	0	**109,95 €**
ES	106,65 € + 0	1	106,65 €	0	0	0	**106,65 €**
UK	£94.56 + £12.00	0,838	127,12 €	3,00 €	29,93 €	2,00 €	**162,05 €**
US	$88.36 + $24.99	1,123	100,90 €	3,00 €	23,90 €	2,00 €	**129,80 €**

Nos dois casos fora da UE, o valor base fica abaixo do limite de 150 €, pelo que se aplica o direito fixo de 3 €.

O exemplo mostra o valor do cálculo para a comparação. A loja US apresenta o preço de prateleira mais baixo (88,36 $), e a loja UK um preço próximo do português, mas, depois de somados os portes, os direitos, o IVA de importação e o desalfandegamento, ambas ficam mais caras do que as lojas da UE: a Mochila custa 162,05 € quando comprada no Reino Unido, contra 106,65 € em Espanha, a opção mais barata. A identificação da melhor loja é tratada na secção 4.6.

4.6 Melhor loja, poupança e alertas

A última fase das transformações transforma os custos totais em informação útil para o utilizador, identificando qual a loja mais barata para cada produto, quanto se poupa face à compra em Portugal e que clientes devem ser avisados por terem atingido o seu preço-alvo. A Figura 13mostra a zona correspondente do *workflow*.

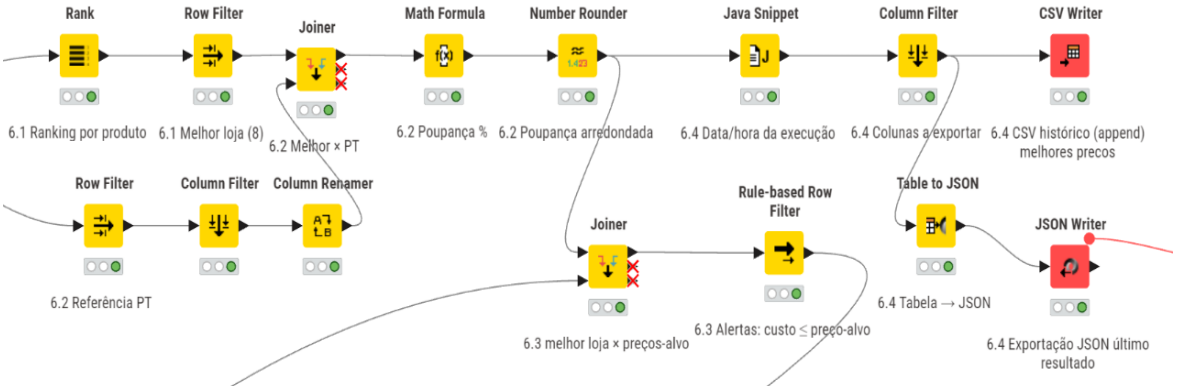


Figura 13 Determinação da melhor loja, cálculo da poupança, alertas e exportação dos resultados.

Melhor loja. O nó Rank ordena as lojas de cada produto pelo custo total (custo_total_eur), por ordem crescente, com agrupamento por EAN. Um Row Filter mantém apenas as linhas com posição 1, obtendo a melhor loja de cada um dos 8 produtos.

Poupança. Num ramo paralelo, um Row Filter seleciona as linhas da loja PT, que servem de referência, e um Joiner associa esse custo à melhor loja de cada produto. A poupança é calculada num nó Math Formula, como (custo PT − melhor custo) / custo PT × 100, e arredondada a duas casas decimais por um Number Rounder. Quando a melhor loja é a própria loja PT, a poupança é 0 %. A Tabela 13 apresenta o resultado de uma execução.

Tabela 13 Melhor loja e poupança por produto (execução de 22 de outubro de 2026).

Produto	Melhor loja	Custo total	Poupança face à loja PT
SSD SanDisk Plus 1TB	ES	104,64 €	6,80 %
Monitor Acer SB220Q	ES	117,50 €	6,67 %
Mochila Fjallraven	ES	106,65 €	3,00 %
Restantes 5 produtos	PT	—	0 %

Este resultado não é fixo. Os preços da loja US variam a cada 10 segundos e, quando o *stream* gera uma promoção relâmpago (−15 %), a loja US pode passar a ser a mais barata num dos produtos, sabendo ainda que as taxas de câmbio também mudam diariamente. Por isso, a melhor loja é recalculada em cada execução.

Alertas. Um Joiner associa a melhor loja de cada produto aos preços-alvo válidos, através do EAN, e um Rule-based Row Filter mantém apenas as linhas que cumprem a regra $custo_total_eur$ <= $preco_alvo_eur$, isto é, aquelas em que o melhor custo total já está ao nível do preço que o cliente pretende pagar. A Tabela 14 mostra três casos representativos dos 6 preços-alvo válidos.

Tabela 14 Comparação entre o melhor custo total e o preço-alvo de três clientes.

Cliente	Produto (melhor loja)	Melhor custo	Preço-alvo	Alerta
Ana	SSD SanDisk Plus 1TB (ES)	104,64 €	105,00 €	Sim
Marta	Disco Externo WD Elements 2TB (PT)	62,72 €	65,00 €	Sim
João	Disco WD 4TB Gaming PS4	110,58 €	110,50 €	Não

Com os dados de teste são gerados 2 alertas, apresentados na Figura 14. O caso do João mostra que a regra é exata: o melhor custo fica apenas 8 cêntimos acima do preço-alvo e, por isso, não é gerado alerta. O preço-alvo do Rui não chega a esta fase, por ter sido rejeitado na validação (e-mail inválido, secção 4.3); por isso, nunca gera alerta.

Rows: 2 | Columns: 5 Sorting Filter

#	RowID	titulo String	loja String	custo_total_eur Number (Float)	email String	preco_alvo_eur Number (Float)
1	Row2...	Disco Externo WD Elements 2TB	PT	62.72	marta.lopes@exemplo.pt	65
2	Row1...	SSD SanDisk Plus 1TB	ES	104.64	ana.silva@exemplo.pt	105

Figura 14 Alertas gerados: clientes cujo preço-alvo foi atingido.

O envio dos e-mails e a regra que impede o envio repetido do mesmo alerta nas últimas 24 horas são tratados como *jobs* no capítulo 5.

Exportação. Por fim, um nó Java Snippet acrescenta a data e a hora da execução, e os resultados são exportados para a pasta data/output em dois formatos: o CSV Writer acrescenta as linhas a melhores_precos.csv (modo *append*), mantendo o histórico de todas as execuções, e o JSON Writer grava melhores_precos_0.json apenas com o último resultado, num formato pronto a ser consumido por outras aplicações.

5. Jobs

Enquanto as transformações descritas no capítulo 4 tratam os dados, os *jobs* executam ações com o resultado obtido: carregar os dados na base de dados (BD), enviar os alertas por e-mail, exportar e transferir ficheiros, registar cada execução e agendar o processo. Neste trabalho, os *jobs* são implementados em duas ferramentas: no KNIME, no final do mesmo *workflow* das transformações, e no Node-RED, que gera os dados da loja US e lança o *workflow* periodicamente. A Tabela 15 resume os *jobs* e a secção em que cada um é descrito.

Tabela 15 Jobs implementados, ferramenta e secção correspondente.

Job	**Ferramenta**	**Função**	**Secção**
Carregamento na BD	KNIME	grava dados de referência (*upsert*) e factos (só inserção) no PostgreSQL	5.1
Alertas por e-mail	KNIME	envia os alertas novos por SMTP (*Simple Mail Transfer Protocol*), sem repetições em 24 horas	5.2
Exportação e SFTP	KNIME	envia os ficheiros de resultados para um servidor remoto por SFTP (*SSH File Transfer Protocol*)	5.3
Erros e *logs*	KNIME	regista cada execução, com sucesso ou erro, sem interromper o processo	5.4
Conversão e *stream*	Node-RED	converte a configuração YAML em JSON e simula os preços da loja US	5.5
Agendamento	Node-RED + NodePit Batch	executa o *workflow* sem interface, de hora a hora	5.6

O *workflow* corre do início ao fim de hora a hora, lançado pelo Node-RED em modo *batch*, ou manualmente, com a opção *Execute all* do KNIME. A ordem entre os *jobs* é garantida por ligações de variáveis de fluxo (*flow variables*): um *job* só começa quando o anterior termina, o que assegura, por exemplo, que as lojas e os produtos existem na BD antes de serem gravados os preços que os referenciam. As falhas no servidor remoto ficam registadas, em vez de interromperem o processo. A Figura 15 apresenta a sequência dos *jobs*.

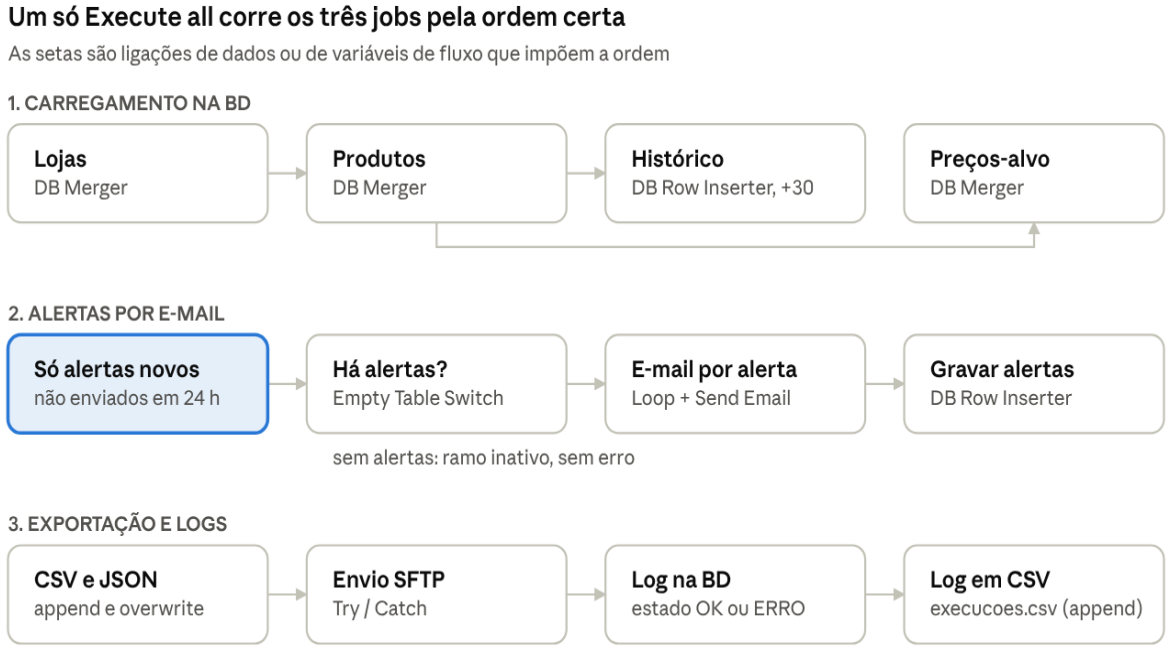


Figura 15 Sequência dos jobs em cada execução; a destaque, a regra das 24 horas.

A regra das 24 horas, assinalada a destaque, é descrita na secção 5.2 e impede que o mesmo alerta seja enviado várias vezes quando o *workflow* corre de hora a hora.

5.1 Carregamento na base de dados

Os resultados das transformações são carregados numa base de dados (BD) PostgreSQL, que corre num contentor Docker e à qual o KNIME acede através do nó PostgreSQL Connector, por JDBC (*Java Database Connectivity*). O modelo de dados, apresentado na Figura 16, distingue três tipos de tabelas: as **tabelas de referência** (lojas, produtos, preços-alvo, câmbios e IVA), que descrevem entidades e podem ser atualizadas; as **tabelas de factos** (historico_precos e alertas), que registam acontecimentos e apenas crescem; e as **tabelas de rastreabilidade** (precos_brutos, rejeitados, logs_execucao e auditoria), que guardam a informação necessária para auditar o processo.

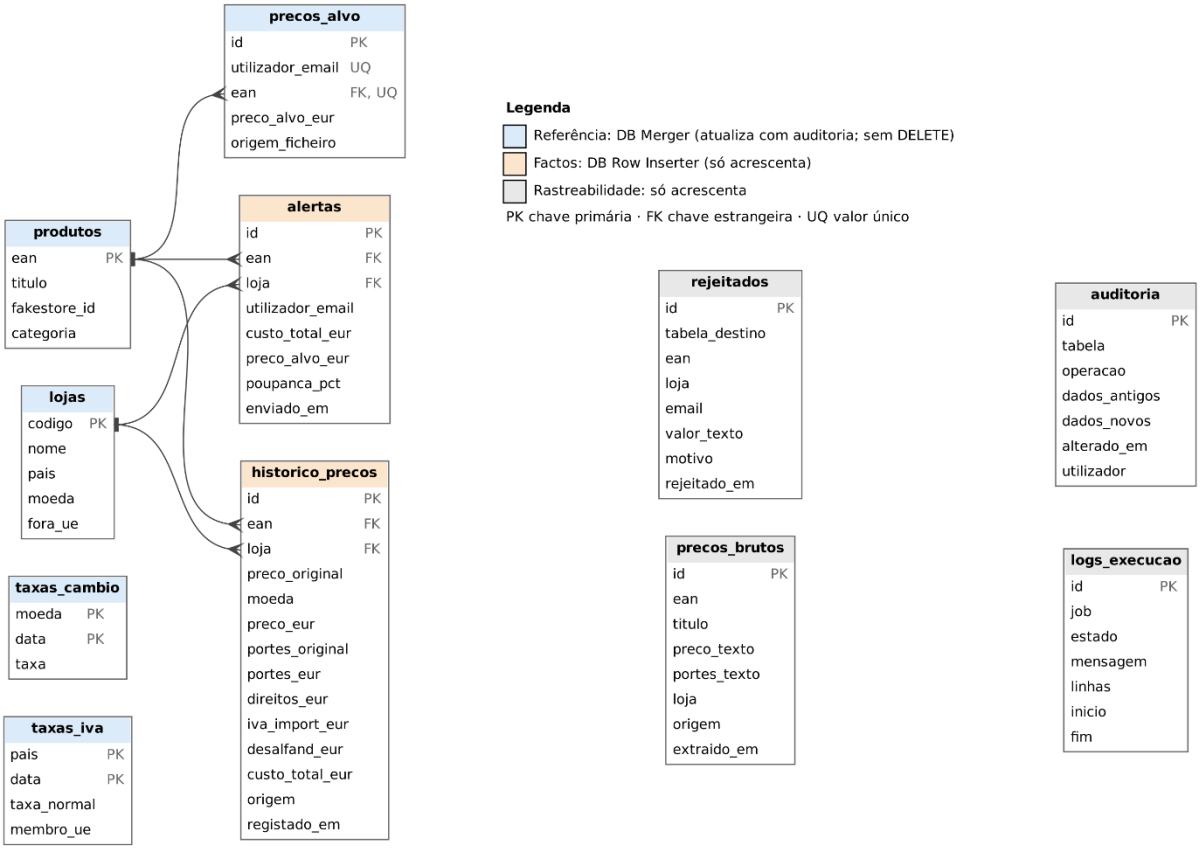


Figura 16 Modelo de dados da base de dados PostgreSQL.

Cada tipo de tabela é carregado com o nó adequado ao seu comportamento, como resume a Tabela 16.

Tabela 16 Nós de carregamento e comportamento de cada tabela.

Tabela	**Nó KNIME**	**Comportamento**
lojas, produtos, precos_alvo	DB Merger	atualiza a linha se já existe e insere-a se é nova (*upsert*); cada alteração fica registada na auditoria
taxas_cambio, taxas_iva	DB Merger, chave (moeda ou país, data)	uma linha por dia, o que mantém o histórico das taxas
historico_precos	DB Row Inserter	só acrescenta: +30 linhas por execução
precos_brutos, rejeitados	DB Row Inserter	só acrescenta (+35 e +6 linhas por execução), para auditoria da limpeza
alertas, logs_execucao	DB Row Inserter / DB SQL Executor	só acrescenta

As chaves estrangeiras (FK, *foreign keys*) obrigam a que as lojas e os produtos existam antes de serem gravados os preços, os preços-alvo e os alertas que os referenciam. A cadeia de variáveis de fluxo lojas → produtos → histórico e preços-alvo garante essa ordem (Figura 17). Num teste, um preço-alvo com o EAN 12345 foi recusado pela própria BD, com a violação da restrição precos_alvo_ean_fkey, o que confirma a integridade referencial. Em conjunto com as regras CHECK descritas na secção 4.3, a BD funciona como uma segunda barreira de validação, independente do KNIME.

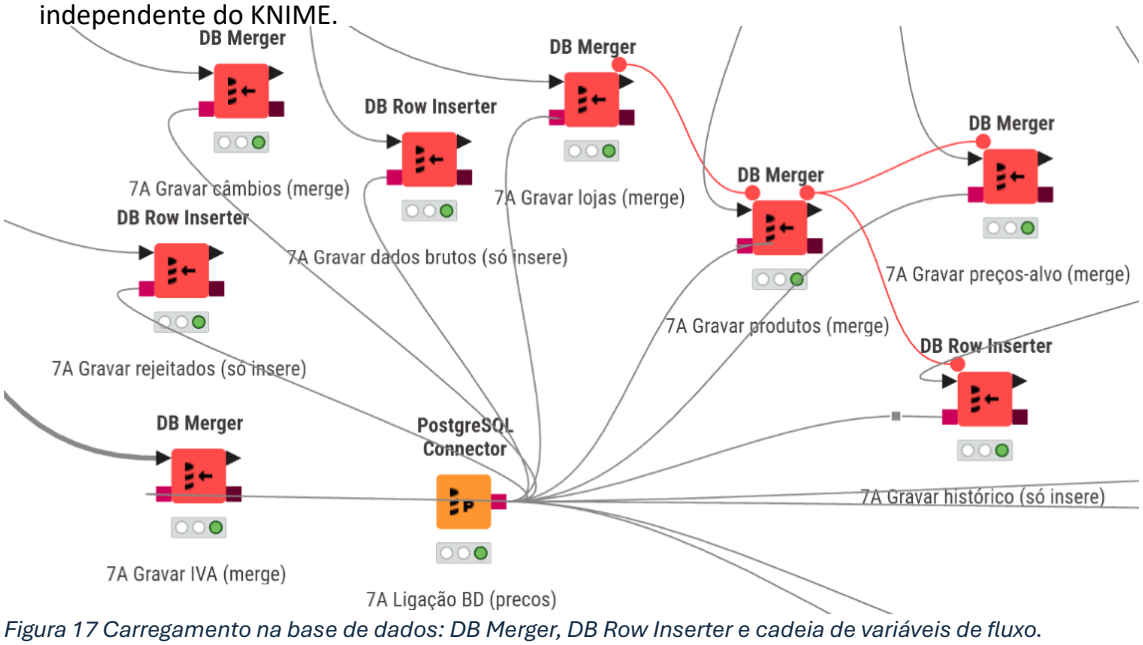


Figura 17 Carregamento na base de dados: DB Merger, DB Row Inserter e cadeia de variáveis de fluxo.

A regra de que os dados nunca são eliminados está garantida na própria BD, e não no KNIME, porque o KNIME não permite proibir nós de eliminação. Assim, a proteção funciona para qualquer programa que se ligue à BD. O Excerto 3 mostra a função que recusa a operação, a sua aplicação às tabelas de factos e o *event trigger* que impede a remoção de tabelas.

```
-- Excerto 3 – Proteção contra a eliminação de dados (src/sql/schema.sql)

CREATE OR REPLACE FUNCTION fn_bloquear() RETURNS trigger AS $$

BEGIN

    RAISE EXCEPTION 'Operação % proibida na tabela %: os dados deste sistema nunca são eliminados nem alterados.',

        TG_OP, TG_TABLE_NAME;

END;

$$ LANGUAGE plpgsql;

-- Tabelas de factos e registos: só se acrescentam linhas

DO $$

DECLARE t TEXT;

BEGIN

    FOREACH t IN ARRAY ARRAY['historico_precos','alertas','logs_execucao','precos_brutos','rejeitados','auditoria'] LOOP

        EXECUTE format('CREATE OR REPLACE TRIGGER trg_%s_so_acrescentar BEFORE UPDATE OR DELETE ON %I FOR EACH ROW EXECUTE FUNCTION fn_bloquear()', t, t);

        EXECUTE format('CREATE OR REPLACE TRIGGER trg_%s_sem_truncate BEFORE TRUNCATE ON %I FOR EACH STATEMENT EXECUTE FUNCTION fn_bloquear()', t, t);

    END LOOP;

END;

$$;

-- Impede também apagar tabelas inteiras

CREATE OR REPLACE FUNCTION fn_bloquear_drop() RETURNS event_trigger AS $$

BEGIN

    RAISE EXCEPTION 'Comando % proibido: as tabelas deste sistema nunca são eliminadas.', TG_TAG;

END;

$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER trg_sem_drop ON ddl_command_start

    WHEN TAG IN ('DROP TABLE', 'DROP SCHEMA')

    EXECUTE FUNCTION fn_bloquear_drop();
```

Nas tabelas de referência, a atualização é permitida, porque o DB Merger precisa dela, mas o DELETE e o TRUNCATE continuam proibidos, e um *trigger* (função fn_auditar) guarda na tabela auditoria a versão anterior e a nova de cada linha alterada. O *event trigger* protege também contra opções do próprio KNIME que apagam tabelas, como *Remove existing table* do nó DB Writer.

A eficácia da proteção foi verificada diretamente no PostgreSQL (Figura 18). Dentro de uma transação, por segurança, um DELETE sobre historico_precos foi recusado pela função fn_bloquear, e um DROP TABLE sobre alertas foi recusado pelo *event trigger*, através da função fn_bloquear_drop. Em ambos os casos, a operação terminou com erro e nenhum dado foi afetado.

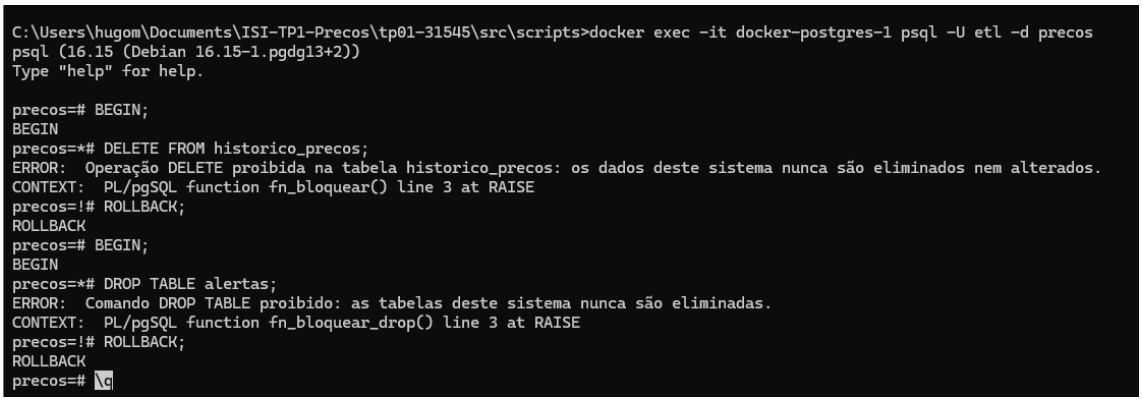


Figura 18 Tentativas de DELETE e DROP TABLE recusadas pela base de dados.

5.2 Alertas por e-mail, sem repetições

Os alertas identificados na secção 4.6 têm de ser comunicados aos clientes por e-mail. Como o *workflow* é executado de hora a hora, um envio direto faria com que o mesmo cliente recebesse o mesmo alerta em todas as execuções, enquanto o preço se mantivesse abaixo do seu preço-alvo. Este *job* envia, por isso, apenas os alertas novos, isto é, os que não foram enviados nas últimas 24 horas. A janela de 24 horas é intencional: se no dia seguinte o preço continuar abaixo do alvo, o cliente volta a ser avisado, uma única vez. A Figura 19 mostra a zona correspondente do *workflow*.

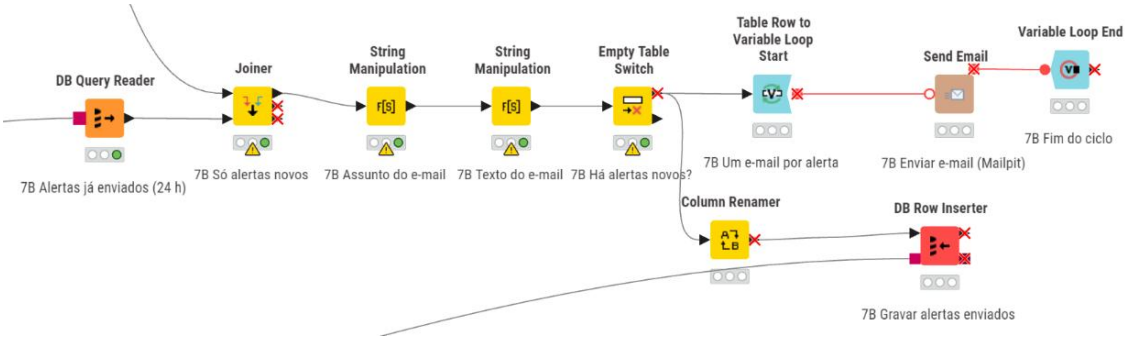


Figura 19 Job de alertas por e-mail, com o ramo de envio inativo por não existirem alertas novos.

O *job* segue os passos seguintes:

1. Um nó DB Query Reader lê da tabela alertas os pares (e-mail, EAN) enviados nas últimas 24 horas (Excerto 4).
2. Um Joiner compara os alertas atuais com esse resultado e mantém apenas as linhas sem correspondência (*left unmatched rows*), ou seja, os alertas ainda não enviados.
3. Um nó String Manipulation compõe o assunto e o texto de cada e-mail.
4. Um nó Empty Table Switch verifica se existem alertas novos, e quando não existem, o ramo de envio fica inativo, sem gerar erro.
5. O ciclo Table Row To Variable Loop Start → Send Email → Variable Loop End envia um e-mail por alerta. Em cada iteração, o destinatário, o assunto e o texto são passados ao nó Send Email através de variáveis de fluxo.
6. Um nó DB Row Inserter grava os alertas enviados na tabela alertas, apenas depois de terminado o ciclo.

```sql
-- Excerto 4 – Alertas já enviados nas últimas 24 horas (nó DB Query Reader)
SELECT utilizador_email AS email, ean
FROM alertas
WHERE enviado_em > NOW() - INTERVAL '24 hours'
```

Duas decisões resultaram de problemas encontrados durante os testes. O nó Empty Table Switch foi acrescentado porque, sem alertas novos, o ciclo executava uma iteração com valores vazios e o servidor de e-mail recusava o envio por endereço inválido (erro 553). A gravação na tabela alertas só depois do ciclo garante que um alerta apenas é registado se o e-mail tiver sido efetivamente enviado ,mas se o envio falhar, o alerta continua a ser considerado novo e é tentado na execução seguinte.

Os e-mails são enviados por SMTP para o Mailpit, um servidor de e-mail de teste que corre num contentor Docker (porta 1025) e apresenta as mensagens recebidas numa interface *web* (porta 8025). Esta opção permite demonstrar o envio sem enviar e-mails reais para endereços fictícios. Para usar um servidor real, basta alterar o servidor, a porta e as credenciais no nó Send Email. Cada e-mail tem o assunto *[«…»]* e indica o produto, a loja mais barata, o custo total e o preço-alvo do cliente (Figura 20).

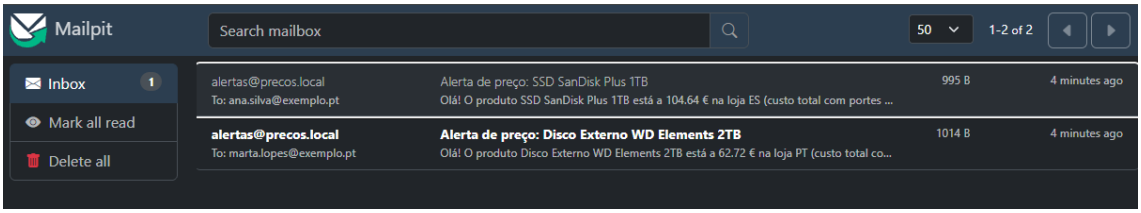


Figura 20 E-mails de alerta recebidos no Mailpit.

Na primeira execução foram enviados 2 e-mails, à Ana e à Marta. Nas execuções seguintes, dentro das 24 horas, o Joiner devolve 0 linhas, o ramo de envio fica inativo (Figura 19) e nenhum e-mail é repetido.

5.3 Exportação e transferência por SFTP

Os resultados de cada execução são exportados para a pasta data/output em dois ficheiros, conforme descrito na secção 4.6: melhores_precos.csv, em modo *append*, que acumula a melhor loja de cada produto em todas as execuções, com data e hora, e melhores_precos_0.json, em modo *overwrite*, que contém apenas o resultado mais recente. Este *job* envia esses ficheiros para um servidor remoto, simulando a entrega dos resultados a um sistema externo, como um parceiro comercial ou outra aplicação.

A transferência é feita por SFTP, um protocolo que transfere ficheiros através de uma ligação SSH cifrada. O servidor corre num contentor Docker (imagem atmoz/sftp), acessível em localhost, na porta 2222, com o utilizador etl e a pasta de destino /upload. No KNIME, o nó SSH Connector abre a ligação e o nó Transfer Files envia todos os ficheiros da pasta de origem, indicada por um caminho relativo ao *workflow* (knime://knime.workflow/data/output), o que mantém o projeto portável (Figura 21).

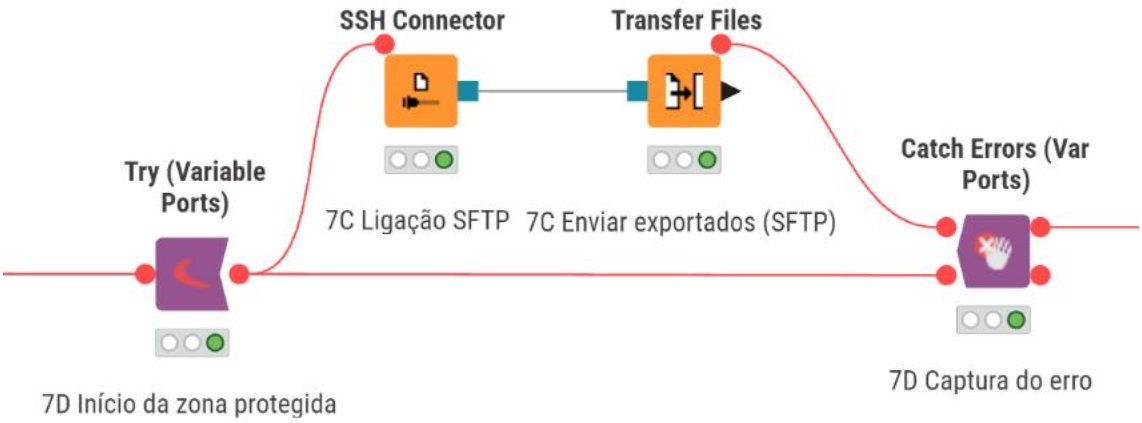


Figura 21 Job de transferência por SFTP, protegido por um bloco Try/Catch.

A Tabela 17 resume a configuração do nó Transfer Files.

Tabela 17 Configuração do nó Transfer Files.

Opção	**Valor**	**Justificação**
Tipo de origem	*Files in folders*	envia todos os ficheiros de data/output (2 ficheiros)
Delete source files / folders	desativada	os ficheiros locais nunca são apagados
Destino	/upload	pasta do utilizador etl no servidor
Create missing folders	ativada	cria a pasta de destino se não existir
Overwrite policy	*Overwrite*	o servidor fica sempre com a versão mais recente de cada ficheiro

A política *Overwrite* não implica perda de informação, uma vez que o ficheiro CSV local é acumulado em modo *append*, a cópia enviada contém sempre o histórico completo de todas as execuções. Em coerência com a regra de que os dados nunca são eliminados, a opção que apaga os ficheiros de origem após o envio não foi ativada, pelo que os ficheiros permanecem em data/output. A Figura 22 mostra os ficheiros no servidor após uma execução.

```
C:\Users\hugom\Documents\ISI-TP1-Precos\tp01-31545\src\scripts>  docker exec docker-sftp-1 ls -l /home/etl/upload
total 20
-rw-r--r-- 1 etl users 14305 Oct 22 10:34 melhores_precos.csv
-rw-r--r-- 1 etl users  1742 Oct 22 10:34 melhores_precos_0.json
```

Figura 22 Ficheiros recebidos na pasta /upload do servidor SFTP.

Como o servidor remoto pode estar indisponível, este *job* está protegido por um bloco Try/Catch: uma falha na ligação fica registada como erro no *log*, mas não impede a conclusão do *workflow* (secção 5.4). Neste protótipo, as credenciais de acesso estão definidas na configuração do nó SSH Connector, numa versão em produção, deveriam ser lidas de um nó Credentials Configuration ou de variáveis de ambiente.

5.4 Tratamento de erros e logs

Um processo automático, executado de hora a hora sem supervisão, tem de cumprir dois requisitos: deixar um registo de cada execução e não ser interrompido por falhas em serviços externos. O serviço mais exposto a falhas é o servidor SFTP, por ser um sistema remoto. Por isso, o bloco de transferência (secção 5.3) está entre um nó Try (Variable Ports) e um nó Catch Errors (Var Ports): se o servidor não responder, o erro é capturado, o *workflow* continua e a execução fica registada com o estado ERRO e a mensagem produzida pelo KNIME.

No fim de cada execução, um nó DB SQL Executor grava uma linha na tabela logs_execucao, e um nó DB Query Reader seguido de um CSV Writer acrescenta essa linha ao ficheiro data/logs/execucoes.csv. O ficheiro permite consultar o histórico das execuções sem acesso à BD e acompanha o projeto (Figura 23).

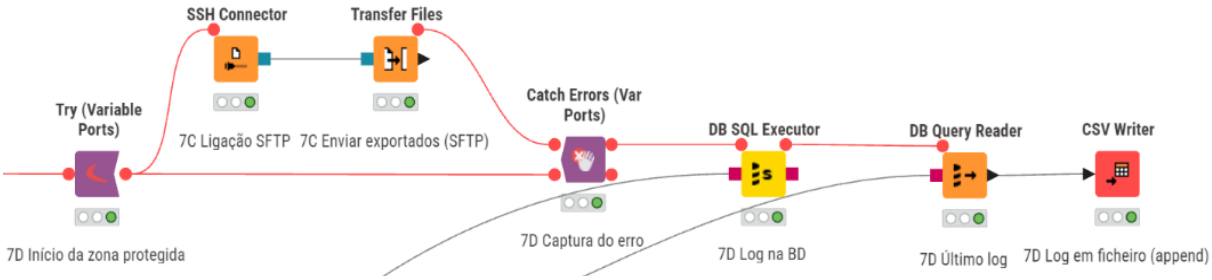


Figura 23 Registo da execução na base de dados e no ficheiro de log.

O estado da execução é decidido no próprio SQL, a partir da variável FailingNodeMessage produzida pelo nó Catch Errors (Excerto 5). Quando não há erro, a variável fica vazia ou com o valor none. A mensagem é delimitada com msg … msg (*dollar quoting* do PostgreSQL), para que eventuais aspas no texto do erro não invalidem a instrução.

```
-- Excerto 5 – Registo da execução com estado OK ou ERRO (nó DB SQL Executor)
INSERT INTO logs_execucao (job, estado, mensagem, linhas, inicio)
SELECT 'tp1_pipeline',
       CASE WHEN m IN ('', 'none') THEN 'OK' ELSE 'ERRO' END,
       CASE WHEN m IN ('', 'none') THEN 'Execução concluída: BD, e-mails e SFTP'
            ELSE 'Falha no SFTP: ' || m END,
       (SELECT COUNT(*) FROM historico_precos WHERE registado_em > NOW() - INTERVAL '15 minutes'),
       NOW() - INTERVAL '1 minute'
FROM (SELECT trim($msg$ $${SFailingNodeMessage}$$ $msg$) AS m) v;
```

As colunas *linhas* e *inicio* são aproximações: a primeira conta os preços gravados nos últimos 15 minutos e a segunda estima o início da execução um minuto antes do registo. São suficientes para acompanhar o funcionamento do sistema, mas uma versão futura poderia registar o início real numa variável de fluxo, no arranque do *workflow*.

A Figura 24 mostra o painel de *logs* do Grafana, onde as execuções com erro se destacam a vermelho.

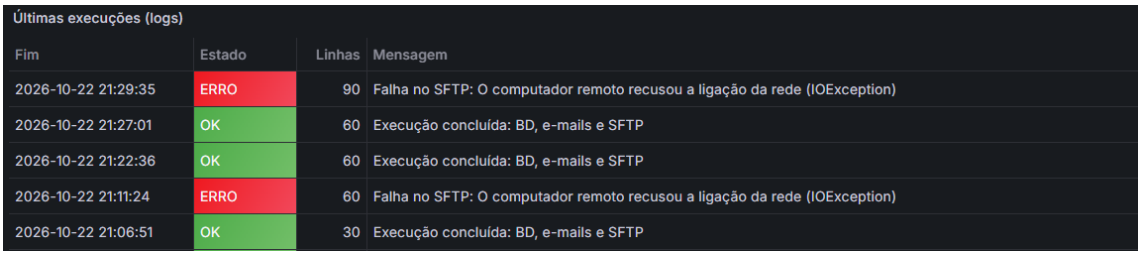


Figura 24 Painel de logs no Grafana, com execuções OK e ERRO.

A Tabela 18 resume os testes realizados ao longo do trabalho para validar o comportamento do sistema.

Tabela 18 Testes realizados e resultados obtidos.

Teste	**Resultado esperado e obtido**	**Ver**
Execução completa, com todos os serviços ativos	estado OK; +35 linhas em precos_brutos, +6 em rejeitados, +30 em historico_precos	secção 4
Linhas com erros propositados (EAN, preço em falta, preço negativo, e-mail)	4 preços e 2 preços-alvo rejeitados, com o motivo	Figura 6, Tabela 8
INSERT com preço de −10 € diretamente na BD	recusado (chk_precos_positivos)	secção 4.3
Segunda execução dentro de 24 horas	0 e-mails novos; ramo de envio inativo	Figura 19 Job de alertas por e-mail, com o ramo de envio inativo por não existirem alertas novos.
DELETE, TRUNCATE e DROP TABLE	recusados pela BD	Figura 18
Execução em *batch* e agendada	código de saída 0	secção 5.6

A proteção por Try/Catch cobre apenas o servidor SFTP. Uma falha nas APIs externas (Frankfurter, VATComply ou Fake Store API) interrompe o *workflow* sem que a execução fique registada. Alargar este mecanismo às restantes fontes é uma das melhorias identificadas no capítulo 7.

5.5 Jobs no Node-RED

O Node-RED é uma ferramenta de programação visual orientada a eventos, em que cada fluxo é composto por nós ligados entre si e pode ser disparado no arranque, a intervalos regulares ou por um pedido HTTP. Estas características tornam-no adequado para gerar dados contínuos e para executar tarefas periódicas. Neste trabalho, o Node-RED executa três jobs: a conversão do ficheiro de configuração, a simulação da loja US e o agendamento do workflow do KNIME, descrito na secção 5.6. Todos os fluxos estão guardados no ficheiro src/node-red/flows.json, com caminhos relativos à pasta do projeto, pelo que podem ser importados noutro computador sem alterações.

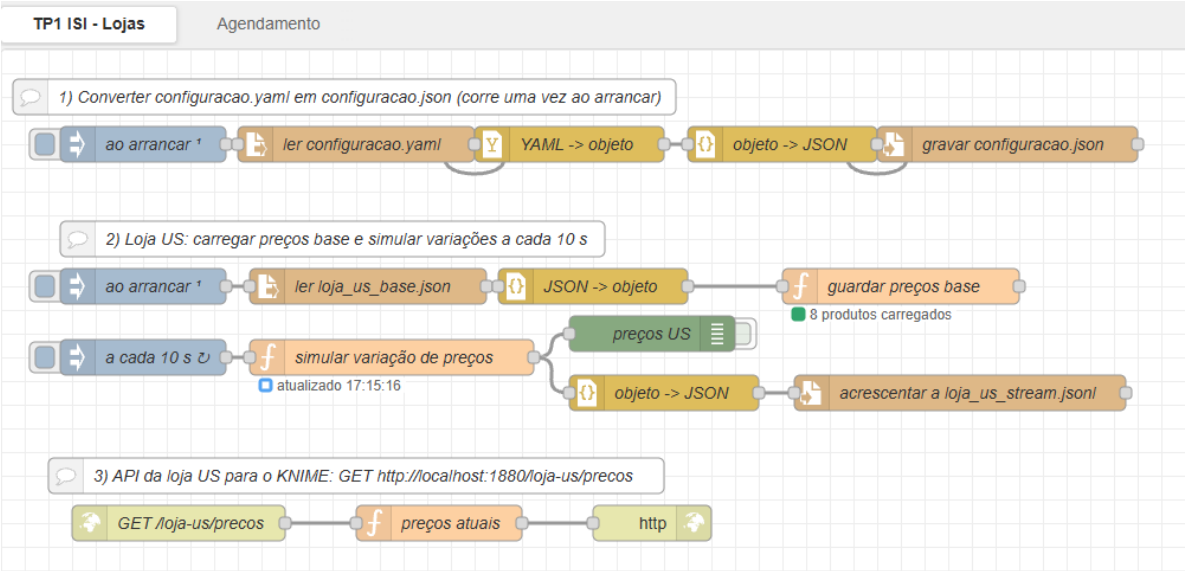


Figura 25 Fluxos do Node-RED: (1) conversão da configuração, (2) simulação da loja US e (3) endpoint para o KNIME.

Conversão YAML → JSON (fluxo 1). A configuração do sistema (país de destino, lojas e regras de importação) é mantida num ficheiro YAML, por ser um formato mais legível para edição manual. No arranque do Node-RED, este fluxo lê o ficheiro configuracao.yaml, converte-o num objeto e grava-o como configuracao.json, que é o ficheiro lido pelo KNIME (secção 4.4).

Simulação da loja US (fluxo 2). A loja dos Estados Unidos é simulada como uma fonte de dados em tempo real. No arranque, os preços base dos 8 produtos são lidos de loja_us_base.json e guardados na memória do fluxo. A cada 10 segundos, um nó *function* atualiza-os (Excerto 6): cada preço varia ±3 % em relação ao preço base e tem 5 % de probabilidade de entrar em promoção relâmpago (−15 %). Os preços são escritos intencionalmente no formato americano ("$104.93" ou, acima de mil dólares, "$1,049.99"), para que o KNIME os tenha de normalizar com expressões regulares (secção 4.2). No fim, o nó constrói a mensagem de saída com os campos ean, name, price, shipping e updated_at, que são os nomes lidos pelo KNIME.

```
javascript

// Excerto 6 – Simulação da variação de preços da loja US (nó function do Node-RED)

const precos = flow.get('precos');

if (!precos) return null;

const agora = new Date().toISOString();

for (const p of precos) {

    let fator = 1 + (Math.random() * 0.06 - 0.03);   // variação normal de ±3%

    if (Math.random() < 0.05) fator = 0.85;           // 5% de hipótese de promoção relâmpago (-15%)

    p.price_usd = Math.round(p.price_usd_base * fator * 100) / 100;

    // formato americano, de propósito, para a Regex limpar no KNIME: $1,049.99

    p.price_text = '$' + p.price_usd.toLocaleString('en-US', { minimumFractionDigits: 2, maximumFractionDigits: 2 });

    p.shipping_text = '$' + p.shipping_usd.toFixed(2);

    p.updated_at = agora;

}

flow.set('precos', precos);

msg.payload = precos.map(p => ({ ean: p.ean, name: p.name, price: p.price_text,

                                 shipping: p.shipping_text, updated_at: p.updated_at }));

node.status({ fill: 'blue', shape: 'ring', text: 'atualizado ' + agora.substring(11, 19) });

return msg;
```

Cada atualização é convertida em JSON e acrescentada ao ficheiro data/input/loja_us_stream.jsonl, no formato JSON Lines (um objeto JSON por linha), que guarda o histórico completo do *stream*. Como este ficheiro cresce continuamente enquanto o Node-RED está ativo (mais de 6300 linhas durante o desenvolvimento), a sua divisão em ficheiros diários é uma das melhorias identificadas no capítulo 7.

Endpoint para o KNIME (fluxo 3). Um terceiro fluxo disponibiliza um *endpoint* HTTP (GET /loja-us/precos): o nó "preços atuais" lê os preços mantidos em memória pela simulação e devolve-os em formato JSON, sendo consultado pelo nó GET Request do KNIME em cada execução (secção 4.1). Desta forma, o KNIME obtém sempre os preços do momento da execução, como mostra a Figura 26.

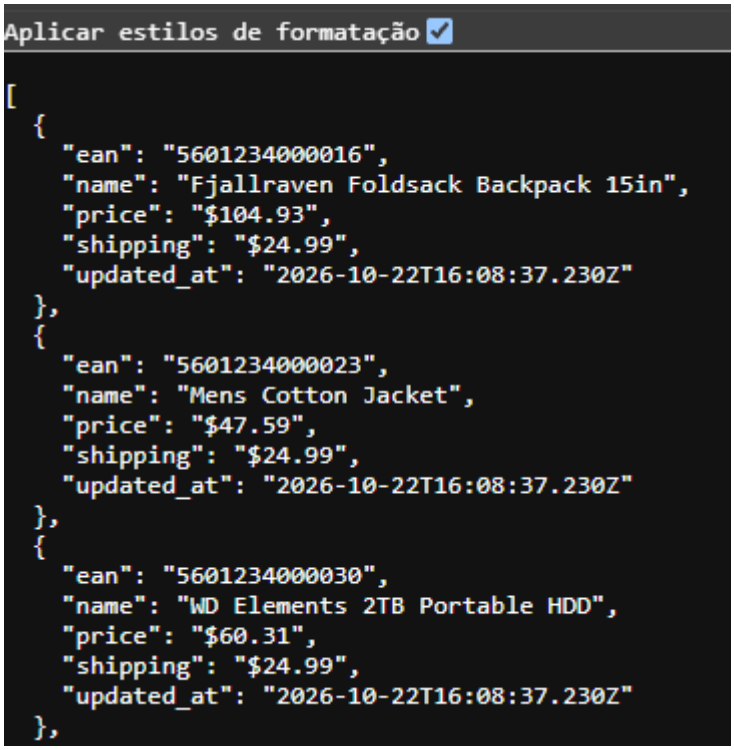


Figura 26 Excerto da resposta do endpoint /loja-us/precos, com os preços no formato americano.

5.6 Execução em batch e agendamento

Para que o sistema funcione sem intervenção, o *workflow* tem de correr sem abrir a interface gráfica do KNIME e de forma periódica. Como a KNIME deixou de disponibilizar o seu Batch Executor a partir da versão 5.9 *[referência]*, a execução sem interface usa a extensão gratuita NodePit Batch, instalada a partir do *update site* da NodePit. O agendamento é feito pelo Node-RED, que lança o *workflow* de hora a hora.

Script de execução. O ficheiro src/scripts/run_pipeline.bat verifica se o Docker está ativo, inicia os contentores caso estejam parados (docker compose up -d) e chama o KNIME com a aplicação do NodePit Batch, indicando o caminho do *workflow* (Excerto 7). O caminho do projeto é calculado a partir da localização do próprio script (%~dp0), e o executável do KNIME é localizado através da variável de ambiente %LOCALAPPDATA% do Windows, pelo que nenhum caminho está escrito de forma absoluta.

bat

```
:: Excerto 7 – Execução do workflow sem interface (src/scripts/run_pipeline.bat)

cd /d "%~dp0..\.."

set "KNIME_EXE=%LOCALAPPDATA%\Programs\KNIME\knime.exe"

docker info >nul 2>&1

if errorlevel 1 ( echo ERRO: o Docker Desktop nao esta a correr. & exit /b 1 )

docker compose -f "src\docker\docker-compose.yml" up -d

start "" /wait "%KNIME_EXE%" -nosplash -consoleLog --launcher.suppressErrors ^

  -data "%CD%" ^

  -application com.nodepit.batch.application.NodePitBatchExecutor ^

  -r --no-save ^

  "%CD%\dataintegration\01_extracao"

set "RC=%ERRORLEVEL%" & echo Fim (codigo %RC%) & exit /b %RC%
```

As opções -r e --no-save reiniciam o *workflow* antes da execução e evitam que o estado da execução seja gravado, mantendo o *workflow* do repositório inalterado. O script devolve o código de saída do NodePit Batch, o que permite identificar a causa de uma falha (Tabela 19).

Tabela 19 Códigos de saída do NodePit Batch.

Código	Significado
0	execução concluída com sucesso
10	parâmetros de entrada inválidos
20	erro ao carregar o *workflow* (por exemplo, o KNIME aberto na mesma pasta de trabalho)
30	erro durante a execução de um nó

Acesso aos ficheiros em modo batch. Durante os testes verificou-se que, em modo *batch*, o KNIME só permite a um *workflow* aceder à sua própria pasta data (*Workflow Data Area*): caminhos relativos para fora do *workflow* falhavam com o código 30. A solução foi criar, dentro do *workflow*, uma pasta data que é uma ligação (*junction*) para a pasta data do projeto. Essa ligação é criada pelo script src/scripts/preparar_ambiente.bat, e todos os nós usam caminhos do tipo knime://knime.workflow/data/..., que funcionam da mesma forma na interface e em *batch*.

Agendamento no Node-RED. No separador "Agendamento", um nó *inject* dispara de hora a hora um nó *exec*, que corre cmd /c ..\scripts\run_pipeline.bat; o resultado, incluindo o código de saída, aparece no painel de *debug* (Figura 27).

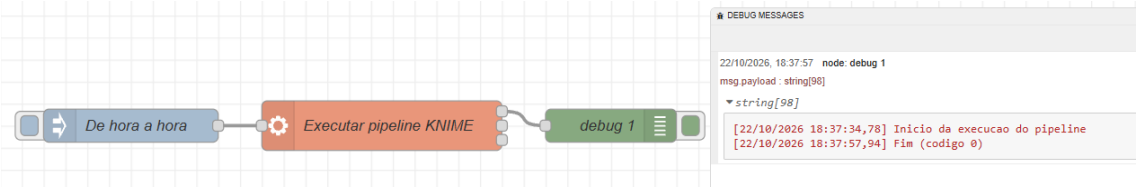


Figura 27 Agendamento do workflow no Node-RED, com o resultado de uma execução.

A Tabela 20 resume os testes da execução automática.

Tabela 20 Testes da execução em batch e do agendamento.

Teste	**Resultado**
run_pipeline.bat com o KNIME fechado	código de saída 0; *log* com estado OK
Disparo manual do agendamento no Node-RED	"Fim (codigo 0)" no *debug* e nova linha em logs_execucao
Agendamento com a interface do KNIME aberta	código 20; o *workflow* não é executado

A principal limitação desta solução é que o modo *batch* usa a mesma pasta de trabalho da interface gráfica, pelo que o KNIME tem de estar fechado quando o agendamento dispara. Esta limitação é retomada no capítulo 7.

6. Vídeo de demonstração

O vídeo de demonstração apresenta o sistema a funcionar de ponta a ponta, em cerca de 7 minutos e meio, e pode ser consultado através do QR Code da Figura 25 ou da ligação abaixo.



Figura 28 QR Code de acesso ao vídeo de demonstração.

Ligação: *[link do YouTube]*

O vídeo mostra, por esta ordem:

1. a infraestrutura em Docker e os fluxos do Node-RED;
2. as fontes de dados, com os erros propositados, e o *endpoint* da loja US a variar de preço;
3. o *workflow* do KNIME, percorrido por zonas, e a sua execução completa;
4. os resultados: registos rejeitados, custo total da Mochila nas quatro lojas, e-mails de alerta no Mailpit e ficheiros no servidor SFTP;
5. as regras de negócio: uma segunda execução sem e-mails repetidos, o registo de erro com o servidor SFTP parado e a recusa de um DELETE pela base de dados;
6. o *dashboard* do Grafana.

7. Conclusão e trabalhos futuros

7.1 Conclusões

O sistema desenvolvido cumpre as três fases de um processo ETL: extrai dados de nove fontes, em quatro formatos de ficheiro (CSV, JSON, XML e YAML), três delas APIs REST e uma delas um *stream* em tempo real, transforma os dados com expressões regulares, regras de validação, junções, cálculos de negócio e carrega o resultado numa base de dados (BD), em ficheiros exportados por SFTP, em alertas por e-mail e num *dashboard*. A execução é automática, de hora a hora, sem interface gráfica.

O resultado mais relevante para o utilizador é mostrar que a loja com o preço de etiqueta mais baixo nem sempre é a mais barata, por exemplo a Mochila Fjallraven custa £94.56 no Reino Unido, mas, depois de somados os portes, os direitos, o IVA de importação e o desalfandegamento, o custo real em Portugal é de 162,05 €, contra 106,65 € em Espanha.

Três decisões revelaram-se determinantes:

- **Proteger os dados na BD e não na ferramenta.** O KNIME não permite proibir nós de eliminação, pelo que a regra de que os dados nunca são eliminados foi implementada com *triggers* e um *event trigger* no PostgreSQL. A proteção funciona seja qual for o programa que se liga à BD.
- **Guardar também o que foi rejeitado.** As tabelas precos_brutos e rejeitados permitem auditar a limpeza em cada execução: das 35 linhas recebidas, 30 são válidas, 1 é duplicada e 4 são rejeitadas, com o respetivo motivo.
- **Testar os casos de falha.** O Empty Table Switch, o bloco Try/Catch e a *junction* da pasta data resultaram de erros reais encontrados durante os testes, o que mostra a importância de testar o sistema em condições adversas e não apenas no caso normal.

Do ponto de vista da aprendizagem, o trabalho permitiu aplicar de forma integrada ferramentas de natureza diferente (ETL visual, programação orientada a eventos, bases de dados relacionais, contentores e visualização), e compreender as dificuldades práticas da automação, como a gestão de caminhos, a execução sem interface e o tratamento de erros.

7.2 Limitações e trabalhos futuros

- **Agendamento com a interface aberta.** O modo *batch* não corre enquanto o KNIME está aberto na mesma pasta de trabalho. Uma versão futura poderia usar uma pasta de trabalho separada para o *batch*, ou uma plataforma de servidor, como o KNIME Business Hub.
- **Dependência de uma extensão externa.** A execução em *batch* depende do NodePit Batch, porque a KNIME descontinuou o seu executor a partir da versão 5.9. Uma alternativa seria o Pentaho Data Integration, que inclui execução por linha de comandos (Kitchen e Pan).
- **Tratamento de erros parcial.** O bloco Try/Catch protege apenas a transferência por SFTP; uma falha nas APIs externas interrompe o *workflow* sem registo. O mecanismo deveria ser alargado a todas as fontes externas.
- **Registo aproximado.** As colunas *linhas* e *inicio* do *log* são estimativas. Numa versão futura deveria contar as linhas efetivamente gravadas e registar a hora real de início.
- **Crescimento do ficheiro do stream.** O ficheiro loja_us_stream.jsonl cresce continuamente e por isso numa futura melhoria ser dividido em ficheiros diários.
- **Regras fiscais simplificadas.** O modelo usa regras fixas (direito de 3 € até 150 €, 2 % acima, IVA de 23 %) e assume que os preços das lojas da UE já incluem IVA, na realidade, as taxas variam com o tipo de produto e as regras de venda à distância.
- **Credenciais na configuração dos nós.** As credenciais do SFTP e da BD estão na configuração dos nós, numa versão em produção deveriam ser lidas de variáveis de ambiente ou de um nó Credentials Configuration.
- **Lojas simuladas e mais destinos.** As lojas são simuladas e numa versão futura poderia ler preços de APIs de lojas reais. Como o país de destino está definido no ficheiro de configuração, o mesmo *pipeline* pode servir outros países da UE alterando apenas esse valor.

Declaração de utilização de Inteligência Artificial

No desenvolvimento deste trabalho foi utilizada uma ferramenta de Inteligência Artificial (IA) generativa como apoio nas seguintes tarefas:

- esclarecimento de dúvidas sobre a configuração das ferramentas (KNIME, Node-RED, Docker, PostgreSQL e Grafana);
- apoio no diagnóstico de erros durante a implementação;
- sugestões de melhorias nas instruções SQL e *scripts* de execução;
- geração do diagrama do modelo de dados e das imagens das equações;
- apoio na estruturação, revisão e correção da redação deste relatório.

8. Referências

atmoz. (s.d.). *atmoz/sftp* [Imagem Docker]. Docker Hub. https://hub.docker.com/r/atmoz/sftp (acedido em 15 de outubro de 2026).

Docker Inc. (s.d.). *Docker Compose overview*. https://docs.docker.com/compose/ (acedido em 15 de outubro de 2026).

Fake Store API. (s.d.). *Fake Store API*. https://fakestoreapi.com/ (15 de outubro de 2026).

Ferreira, L., & Ribeiro, Ó. (2026). *Integração de Sistemas de Informação – Trabalho Prático I: enunciado*. Licenciatura em Engenharia de Sistemas Informáticos.

Frankfurter. (s.d.). *Frankfurter: exchange rates API*. https://www.frankfurter.app/ (acedido em 15 de outubro de 2026).

Grafana Labs. (s.d.). *PostgreSQL data source*. https://grafana.com/docs/grafana/latest/datasources/postgres/ (acedido em 17 de outubro de 2026).

GS1. (s.d.). *How to calculate a check digit manually*. https://www.gs1.org/services/how-calculate-check-digit-manually (acedido em 16 de outubro de 2026).

Hitachi Vantara. (s.d.). *About Kitchen, Pan, and Carte*. Pentaho Documentation. https://docs.pentaho.com/pdia-try-pdia/about-pentaho-workflows/pentaho-data-integration-workflows/evaluate-and-learn-pentaho-data-integration-pdi/about-kitchen-pan-and-carte (acedido em 19 de outubro de 2026).

JSON Lines. (s.d.). *JSON Lines*. https://jsonlines.org/ (acedido em 17 de outubro de 2026).

KNIME AG. (s.d.). *KNIME Documentation*. https://docs.knime.com/ (acedido em 13 de outubro de 2026).

KNIME Forum. (2026, 13 de abril). *Batch execution no longer supported*. https://forum.knime.com/t/batch-execution-no-longer-supported/91773 (acedido em 18 de outubro de 2026).

Mailpit. (s.d.). *Mailpit documentation*. https://mailpit.axllent.org/ (acedido em 18 de outubro de 2026).

NodePit. (s.d.). *NodePit Batch*. https://nodepit.com/product/batch (acedido em 18 de outubro de 2026).

OpenJS Foundation. (s.d.). *Node-RED documentation*. https://nodered.org/docs/ (acedido em 18 de outubro de 2026).

Oracle. (s.d.). *Class Pattern* (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html (acedido em 18 de outubro de 2026).

PostgreSQL Global Development Group. (s.d.). *CREATE TRIGGER*. https://www.postgresql.org/docs/current/sql-createtrigger.html (17 de outubro de 2026).

PostgreSQL Global Development Group. (s.d.). *Event triggers*. https://www.postgresql.org/docs/current/event-triggers.html (17 de outubro de 2026).

VATComply. (s.d.). *VATComply API*. https://www.vatcomply.com/ (acedido em 15 de outubro de 2026).